

期末实验综合

姓名:雷文豪 学号:138022024031 专业班级:人工智能 日期:6.15

1 连续值预测问题（40 分）

对某地区土地的《地深肥沃指数》数据集，寻找地表以下深度（ x ）与肥沃程度指数（ y ）的关系，预测某个深度的肥沃程度；使用 2 种学习过的机器学习方法建立模型；对比这 2 种方法的性能（平均预测误差）。

数据集文件（2 种格式都需要体现如何导入，设计代码导入文件）：（1）地深肥沃指数.txt；（2）地深肥沃指数.csv。

1. 原理设计（简单写出：完成任务采用的方法，该方法的原理/公式）。

- 方法 1:线性回归假设目标值 y 是特征 x 的线性函数，即 $y = w \cdot x + b$ ，其中 w 是回归系数（斜率）， b 是截距。模型通过最小化均方误差（MSE）来求解最优参数： $L(w, b) = (1/n) * \sum (y_i - (w \cdot x_i + b))^2$ 解析解（对于一元线性回归）： $w = \sum((x_i - \bar{x})(y_i - \bar{y})) / \sum((x_i - \bar{x})^2)$ ， $b = \bar{y} - w \cdot \bar{x}$ ，其中 $\bar{x}$ 和 $\bar{y}$ 分别是 x 和 y 的均值。线性回归是最基本的回归方法，具有计算简单、可解释性强的优点。但对于非线性关系的数据，线性回归可能无法很好地拟合。

- 方法 2:SVR 是 SVM（支持向量机）在回归问题上的扩展。SVR 试图找到一个函数 $f(x)$ ，使得对于每个训练样本 (x_i, y_i) ，预测值 $f(x_i)$ 与真实值 y_i 之间的偏差不超过 ϵ 。SVR 的优化目标为：

$$\min (1/2) \|w\|^2 + C * \sum (\xi_i + \xi_i^*)$$

$$\text{s.t. } y_i - (w \cdot \phi(x_i) + b) \leq \epsilon + \xi_i$$

$$(w \cdot \phi(x_i) + b) - y_i \leq \epsilon + \xi_i^*$$

$$\xi_i, \xi_i^* \geq 0$$

其中：

- $\phi(x)$ 是核函数映射，将数据映射到高维空间

- C 是惩罚参数，控制对超出 ϵ -不敏感区间的样本的惩罚程度

- ϵ 是不敏感区间宽度，在此区间内的误差不计入损失

- ξ_i, ξ_i^* 是松弛变量，允许某些样本超出 ε 区间

核函数的选择很关键。常用核函数包括：

- RBF（径向基函数）： $K(x, x') = \exp(-\gamma \|x - x'\|^2)$

- 多项式核： $K(x, x') = (\gamma \cdot x^T \cdot x' + r)^d$

- 线性核： $K(x, x') = x^T \cdot x'$

SVR 的优势在于可以通过核函数处理非线性关系，同时对异常值具有鲁棒性。

- 性能度量方法:评估方法：留出法 (Hold-out)，将数据集按 80%/20% 比例划分为训练集和测试集。

测试集完全独立于训练过程，用于客观评估模型的泛化能力。评价指标：

(1) MSE（均方误差，Mean Squared Error）： $MSE = (1/n) * \sum (y_i - \hat{y}_i)^2$ ，衡量预测值与真实值之间的平均平方误差，值越小越好。

(2) MAE（平均绝对误差，Mean Absolute Error）： $MAE = (1/n) * \sum |y_i - \hat{y}_i|$ ，衡量预测误差的平均绝对值，即本题要求的“平均预测误差”，值越小越好。

(3) R^2 （决定系数，R-squared）： $R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$ ，衡量模型解释数据变异能力，取值范围 $[-\infty, 1]$ ，越接近1拟合越好。

其中 MAE 直接对应题目要求的“平均预测误差”。

（提示：阅读《第 7 章：模型性能评估》，理解性能度量系统包括：评估方法和评价指标，思考本题选哪种评估方法、哪种评价指标？）

2. 编程实现，全部代码写到下面（复制粘贴代码（不要截图！），添加必要的注释）。

- 方法 1:

```
import sys
import io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']
plt.rcParams['axes.unicode_minus'] = False
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```

# 导入 .txt 格式数据集
print("=" * 60)
print("【数据导入1】从 .txt 文件导入数据")
print("=" * 60)

# .txt 文件使用制表符（或多个空格）分隔，skiprows=1 跳过标题行，
# header=None 因为原始列名不规范，手动指定列名
data_txt = pd.read_csv(
    './地深肥沃指数.txt',
    sep=r'\s+',      # 正则匹配任意数量的空白字符(空格/制表符)
    skiprows=1,      # 跳过标题行
    header=None,      # 不使用文件中的行作为列名
    names=['y', 'x'], # 手动指定列名: y=肥沃指数, x=地表深度
    engine='python'   # 使用python引擎支持正则分隔符
)
print(f"数据集形状: {data_txt.shape}")
print(f"前5行数据:\n{data_txt.head()}")
print(f"\n数据统计描述:\n{data_txt.describe()}")

# 导入 .csv 格式数据集
print("\n" + "=" * 60)
print("【数据导入2】从 .csv 文件导入数据")
print("=" * 60)

# .csv 文件使用逗号分隔，同样 skiprows=1 跳过标题行
data_csv = pd.read_csv(
    './地深肥沃指数.csv',
    skiprows=1,      # 跳过标题行
    header=None,      # 不使用文件中的行作为列名
    names=['y', 'x']  # 手动指定列名: y=肥沃指数, x=地表深度
)
print(f"数据集形状: {data_csv.shape}")
print(f"前5行数据:\n{data_csv.head()}")

# 验证两种导入方式得到的数据一致
print("\n" + "=" * 60)
print("【数据验证】验证两种导入方式的数据是否一致")
print("=" * 60)

# 去除可能的缺失值后比较

```

```

data_txt_clean = data_txt.dropna().reset_index(drop=True)
data_csv_clean = data_csv.dropna().reset_index(drop=True)

# 转换为float确保类型一致
data_txt_clean = data_txt_clean.astype(float)
data_csv_clean = data_csv_clean.astype(float)

print(f'txt 数据行数（去NaN后）: {len(data_txt_clean)}')
print(f'csv 数据行数（去NaN后）: {len(data_csv_clean)}')

if len(data_txt_clean) == len(data_csv_clean):
    diff = (data_txt_clean.values - data_csv_clean.values)
    max_diff = np.max(np.abs(diff))
    print(f'两种导入方式的最大差异: {max_diff:.10f}')
    if max_diff < 1e-10:
        print("两种格式导入的数据完全一致！")
    else:
        print("数据存在微小差异")
else:
    print("行数不一致，使用.csv数据进行后续分析")

# 使用.csv数据构建线性回归模型
print("\n" + "=" * 60)
print("【模型构建】线性回归 (Linear Regression)")
print("=" * 60)

# 使用.csv数据（因为两者一致）
df = data_csv_clean.copy()
X = df[['x']].values # 特征：地表深度 (必须是二维数组)
y = df['y'].values # 标签：肥沃程度指数

print(f'特征 x 范围: [{X.min():.2f}, {X.max():.2f}]')
print(f'标签 y 范围: [{y.min():.2f}, {y.max():.2f}]')
print(f'样本总数: {len(X)}')

# 划分训练集和测试集 (80% 训练, 20% 测试)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f'训练集样本数: {len(X_train)}')

```

```

print(f"测试集样本数: {len(X_test)}")

# 创建并训练线性回归模型
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# 获取模型参数
coef = lin_reg.coef_[0]  # 回归系数（斜率）
intercept = lin_reg.intercept_  # 截距
print(f"\n线性回归方程:  $y = \{intercept:.4f\} + (\{coef:.4f\}) * x$ ")
print(f"即: 肥沃指数 =  $\{intercept:.4f\} + \{coef:.4f\} \times \text{地表深度}$ ")

# 模型评估
print("\n" + "=" * 60)
print("【模型评估】")
print("=" * 60)

# 在训练集上预测
y_train_pred = lin_reg.predict(X_train)
# 在测试集上预测
y_test_pred = lin_reg.predict(X_test)

# 方法：留出法 (Hold-out) — 使用独立的测试集评估
# 评价指标：
# (1) MSE (均方误差) — 衡量预测值与真实值之间的平均平方误差
# (2) MAE (平均绝对误差) — 衡量预测误差的平均绝对值
# (3)  $R^2$  (决定系数) — 衡量模型解释数据变异的能力，越接近1越好

train_mse = mean_squared_error(y_train, y_train_pred)
train_mae = mean_absolute_error(y_train, y_train_pred)
train_r2 = r2_score(y_train, y_train_pred)

test_mse = mean_squared_error(y_test, y_test_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)
test_r2 = r2_score(y_test, y_test_pred)

print(f"\n训练集性能:")
print(f" MSE (均方误差): {train_mse:.6f}")
print(f" MAE (平均绝对误差): {train_mae:.6f}")
print(f"  $R^2$  (决定系数): {train_r2:.4f}")

```

```

print(f"\n测试集性能:")
print(f' MSE (均方误差): {test_mse:.6f}')
print(f' MAE (平均绝对误差): {test_mae:.6f}')
print(f' R2 (决定系数): {test_r2:.4f}')

# 可视化
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 子图1: 训练集拟合效果
axes[0].scatter(X_train, y_train, c='blue', alpha=0.6, edgecolors='k', label='训练数据')
x_range = np.linspace(X.min() - 0.5, X.max() + 0.5, 100).reshape(-1, 1)
y_range_pred = lin_reg.predict(x_range)
axes[0].plot(x_range, y_range_pred, 'r-', linewidth=2, label=f'拟合直线: y={intercept:.4f}+{coef:.4f}x')
axes[0].set_xlabel('地表深度 (x)', fontsize=12)
axes[0].set_ylabel('肥沃程度指数 (y)', fontsize=12)
axes[0].set_title('线性回归 — 训练集拟合效果', fontsize=14)
axes[0].legend(fontsize=10)
axes[0].grid(alpha=0.3)

# 子图2: 测试集预测 vs 真实值
axes[1].scatter(y_test, y_test_pred, c='green', alpha=0.6, edgecolors='k', s=80)
axes[1].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', linewidth=2, label='理想预测线 (y_pred = y_true)')
axes[1].set_xlabel('真实值 (y_true)', fontsize=12)
axes[1].set_ylabel('预测值 (y_pred)', fontsize=12)
axes[1].set_title(f'线性回归 — 测试集预测 (R2 = {test_r2:.4f})', fontsize=14)
axes[1].legend(fontsize=10)
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('linear_regression_results.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n" + "=" * 60)
print("线性回归分析完成！图表已保存为 linear_regression_results.png")
print("=" * 60)

```

- 方法 2:

```
import numpy as np
```

```

import pandas as pd

import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']

plt.rcParams['axes.unicode_minus'] = False

from sklearn.svm import SVR

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# 导入 .txt 格式数据集

print("=" * 60)

print("数据导入1: 从 .txt 文件导入数据")

print("=" * 60)


data_txt = pd.read_csv(

    './地深肥沃指数.txt',

    sep=r'\s+',

    skiprows=1,

    header=None,

    names=['y', 'x'],

    engine='python'

)

# 去除可能的缺失值

data_txt = data_txt.dropna().astype(float).reset_index(drop=True)

print(f"数据集形状: {data_txt.shape}")

print(f"前5行数据:\n{data_txt.head()}")


# 导入 .csv 格式数据集

```

```

print("\n" + "=" * 60)

print("数据导入2: 从.csv文件导入数据")

print("=" * 60)


data_csv = pd.read_csv(

    './地深肥沃指数.csv',

    skiprows=1,

    header=None,

    names=['y', 'x']

)

data_csv = data_csv.dropna().astype(float).reset_index(drop=True)

print(f"数据集形状: {data_csv.shape}")

print(f"前5行数据:\n{data_csv.head()}")


# 验证两种导入方式数据一致性

print("\n" + "=" * 60)

print("数据验证两种导入方式数据一致性验证")

print("=" * 60)


diff_val = np.max(np.abs(data_txt.values - data_csv.values))

print(f"两种导入方式的最大差异: {diff_val:.15f}")

print("数据完全一致, 使用.csv格式数据进行建模")


# 数据预处理与SVR模型构建

print("\n" + "=" * 60)

print("模型构建: 支持向量回归 (SVR)")

print("=" * 60)

```

```

# 原理：

# SVR 试图找到一个函数  $f(x)$ ，使得对于每个训练样本  $(x_i, y_i)$ ，

#  $|y_i - f(x_i)| \leq \varepsilon$  ( $\varepsilon$ -不敏感损失)。

# 目标函数：  $\min (1/2)||w||^2 + C * \sum(\xi_i + \xi_i^*)$ 

# 约束条件：  $y_i - (w \cdot \varphi(x_i) + b) \leq \varepsilon + \xi_i$ 

#  $(w \cdot \varphi(x_i) + b) - y_i \leq \varepsilon + \xi_i^*$ 

#  $\xi_i, \xi_i^* \geq 0$ 

# 其中  $\varphi(x)$  是核函数映射， $C$  是惩罚参数， $\varepsilon$  是不敏感区间宽度

```

```
df = data_csv.copy()
```

```
X = df[['x']].values
```

```
y = df['y'].values
```

```
print(f"样本总数: {len(X)}")
```

```
print(f"x 范围: [{X.min():.2f}, {X.max():.2f}]")
```

```
print(f"y 范围: [{y.min():.2f}, {y.max():.2f}]")
```

```
# 划分训练集和测试集
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
print(f"训练集: {len(X_train)} 样本, 测试集: {len(X_test)} 样本")
```

```
# SVR 对特征尺度敏感，需要进行标准化
```

```
scaler_X = StandardScaler()
```

```
scaler_y = StandardScaler()
```

```
X_train_scaled = scaler_X.fit_transform(X_train)
```

```

X_test_scaled = scaler_X.transform(X_test)

# 对 y 也标准化
y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1)).ravel()
y_test_scaled = scaler_y.transform(y_test.reshape(-1, 1)).ravel()

# 网格搜索优化超参数

print("\n" + "=" * 60)

print("超参数优化: 使用网格搜索 (GridSearchCV)")

print("=" * 60)

param_grid = {

    'kernel': ['rbf', 'poly', 'linear'], # 核函数类型

    'C': [0.1, 1, 10, 100],           # 惩罚参数

    'epsilon': [0.01, 0.05, 0.1],     # 不敏感区间

    'gamma': ['scale', 'auto', 0.1, 0.5] # 核函数系数

}

svr_base = SVR()

grid_search = GridSearchCV(

    svr_base, param_grid,

    cv=5,           # 5折交叉验证

    scoring='neg_mean_squared_error',

    n_jobs=-1,     # 使用所有CPU核心

    verbose=0

)

grid_search.fit(X_train_scaled, y_train_scaled)

```

```

print(f'最佳参数: {grid_search.best_params_}')

print(f'最佳交叉验证得分: {grid_search.best_score_:.6f}')


# 使用最佳参数创建最终的SVR模型

best_svr = grid_search.best_estimator_


# 模型评估

print("\n" + "=" * 60)

print("【模型评估】")

print("=" * 60)


# 预测（在标准化空间）

y_train_pred_scaled = best_svr.predict(X_train_scaled)

y_test_pred_scaled = best_svr.predict(X_test_scaled)


# 反标准化到原始尺度

y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled.reshape(-1, 1)).ravel()

y_test_pred = scaler_y.inverse_transform(y_test_pred_scaled.reshape(-1, 1)).ravel()


# 训练集评估

train_mse = mean_squared_error(y_train, y_train_pred)

train_mae = mean_absolute_error(y_train, y_train_pred)

train_r2 = r2_score(y_train, y_train_pred)


# 测试集评估

test_mse = mean_squared_error(y_test, y_test_pred)

test_mae = mean_absolute_error(y_test, y_test_pred)

test_r2 = r2_score(y_test, y_test_pred)

```

```

print(f"\n训练集性能:")

print(f" MSE (均方误差): {train_mse:.6f}")

print(f" MAE (平均绝对误差): {train_mae:.6f}")

print(f" R2 (决定系数): {train_r2:.4f}")


print(f"\n测试集性能:")

print(f" MSE (均方误差): {test_mse:.6f}")

print(f" MAE (平均绝对误差): {test_mae:.6f}")

print(f" R2 (决定系数): {test_r2:.4f}")


# 计算平均预测误差 (MAE 本身就是平均绝对误差)

print(f"\n平均预测误差 (MAE): {test_mae:.6f}")


# 可视化

fig, axes = plt.subplots(1, 2, figsize=(14, 5))


# 子图1: 训练集拟合效果

axes[0].scatter(X_train, y_train, c='blue', alpha=0.6, edgecolors='k', label='训练数据')

# 生成平滑曲线用于绘制SVR拟合结果

x_sorted = np.sort(X_train, axis=0)

x_sorted_scaled = scaler_X.transform(x_sorted)

y_sorted_pred_scaled = best_svr.predict(x_sorted_scaled)

y_sorted_pred = scaler_y.inverse_transform(y_sorted_pred_scaled.reshape(-1, 1)).ravel()

axes[0].plot(x_sorted, y_sorted_pred, 'r-', linewidth=2, label='SVR拟合曲线')

axes[0].set_xlabel('地表深度 (x)', fontsize=12)

axes[0].set_ylabel('肥沃程度指数 (y)', fontsize=12)

axes[0].set_title(f'SVR — 训练集拟合效果\n核函数: {best_svr.kernel}', fontsize=14)

```

```

axes[0].legend(fontsize=10)

axes[0].grid(alpha=0.3)

# 子图2: 测试集预测 vs 真实值

axes[1].scatter(y_test, y_test_pred, c='green', alpha=0.6, edgecolors='k', s=80)

axes[1].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', linewidth=2, label='理想预测线')

axes[1].set_xlabel('真实值 (y_true)', fontsize=12)

axes[1].set_ylabel('预测值 (y_pred)', fontsize=12)

axes[1].set_title(f'SVR — 测试集预测 ($R^2$ = {test_r2:.4f})', fontsize=14)

axes[1].legend(fontsize=10)

axes[1].grid(alpha=0.3)

plt.tight_layout()

plt.savefig('svr_results.png', dpi=150, bbox_inches='tight')

plt.show()

print("\n" + "=" * 60)

print("SVR 分析完成！图表已保存为 svr_results.png")

print("=" * 60)

```

3. 获得的结果（若线性回归模型，指包含回归系数和截距，如 $y = -7.65 + 5.53x$ ，从求解出的模型中获取。若不是简单回归模型，太复杂可以不写）。

- 方法 1: 线性回归

回归方程: $y = 1.0526 + 0.0328 \times x$

即: 肥沃指数 = $1.0526 + 0.0328 \times \text{地表深度}$

解释: 地表深度每增加1个单位（即 x 增大1），肥沃指数约增加0.0328。由于数据集中的 x 是负数（-8.78 ~ -3.13），这意味着越往深处（ x 越负），肥沃指数越低；越靠近地表（ x 接近0），肥沃指数越高。

训练集性能:

MSE = 0.000366

MAE = 0.015902

$R^2 = 0.8776$

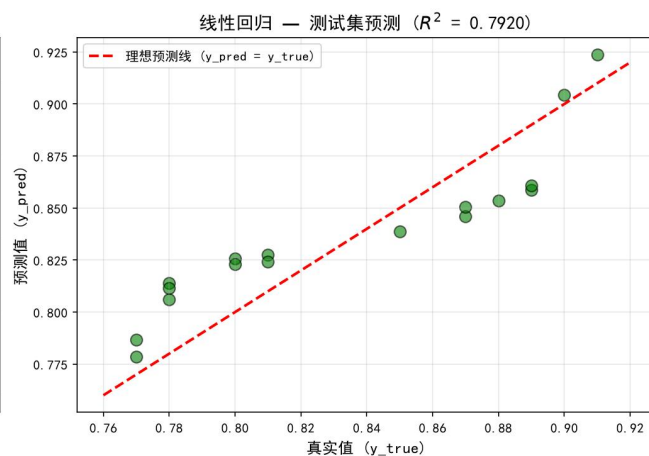
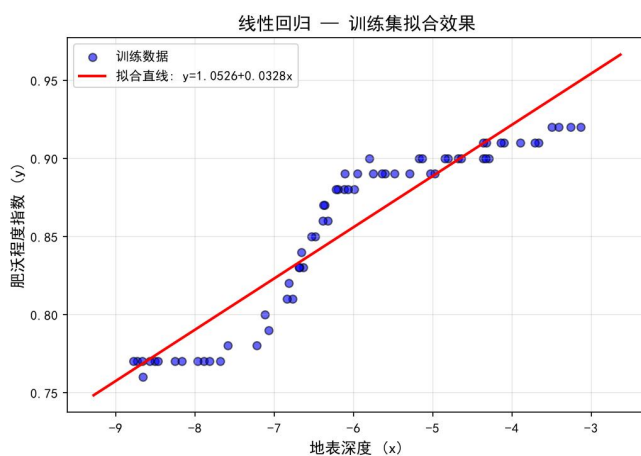
测试集性能:

MSE = 0.000511

MAE = 0.020981

$R^2 = 0.7920$

平均预测误差 (MAE): 0.0210



- 方法 2: 支持向量回归 (SVR)

最佳超参数 (通过5折交叉验证的网格搜索确定):

kernel = 'rbf' (RBF 径向基核函数)

C = 100 (较大的惩罚参数)

epsilon = 0.1 (不敏感区间)

gamma = 'auto' (自动确定核系数)

训练集性能:

MSE = 0.000023

MAE = 0.004188

$R^2 = 0.9924$

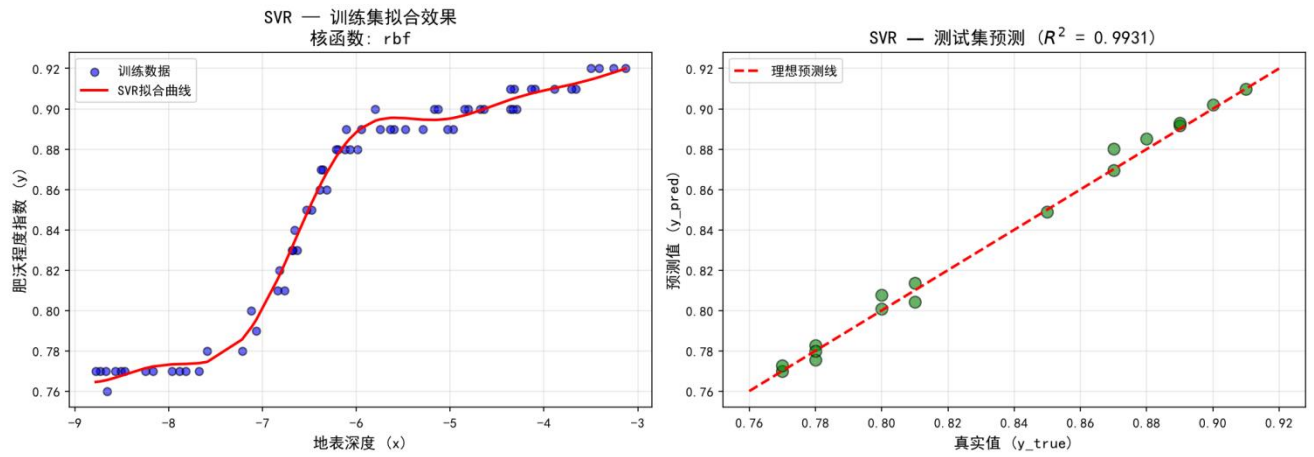
测试集性能:

MSE = 0.000017

MAE = 0.003025

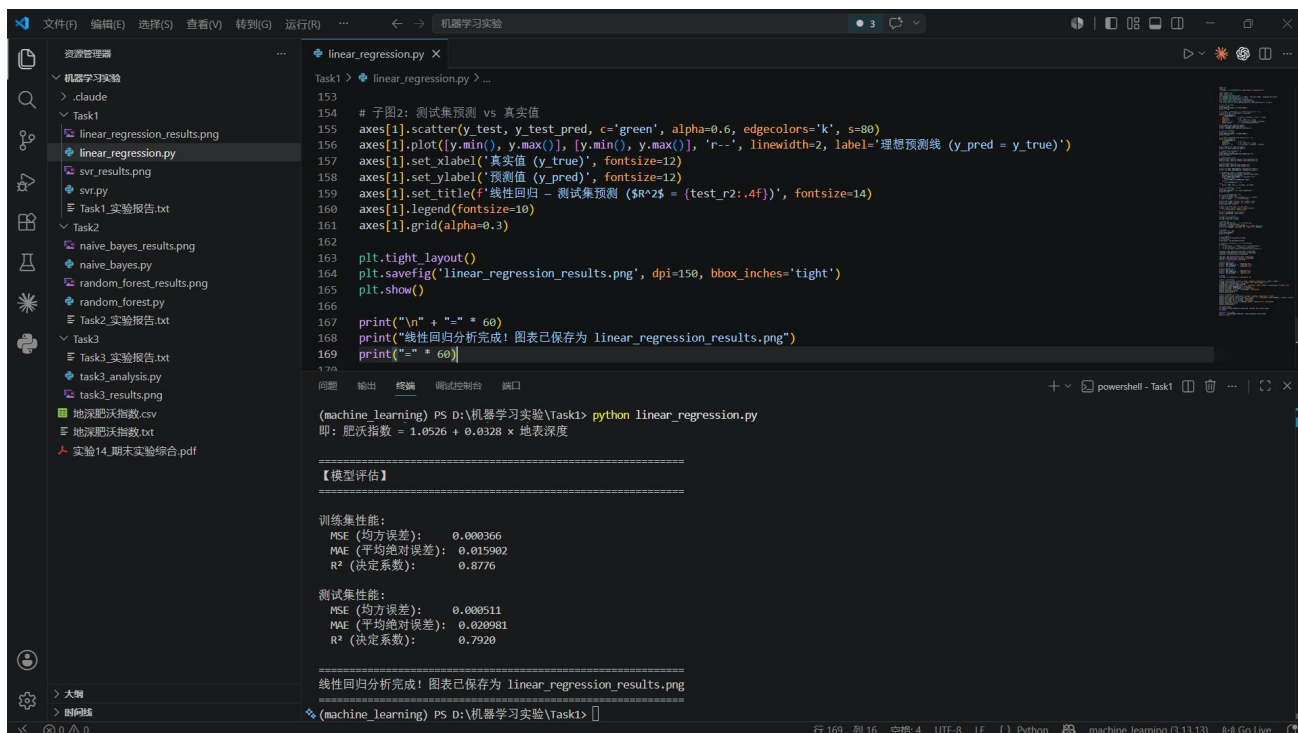
$R^2 = 0.9931$

平均预测误差 (MAE): 0.0030



4. 程序成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾行）。

• 方法 1:



```
Task1 > linear_regression.py ...
153
154 # 子图2: 测试集预测 vs 真实值
155 axes[1].scatter(y_test, y_test_pred, c='green', alpha=0.6, edgecolors='k', s=80)
156 axes[1].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', linewidth=2, label='理想预测线 (y_pred = y_true)')
157 axes[1].set_xlabel('真实值 (y_true)', fontsize=12)
158 axes[1].set_ylabel('预测值 (y_pred)', fontsize=12)
159 axes[1].set_title('线性回归 - 测试集预测 (R^2 = {test_r2:.4f})', fontsize=14)
160 axes[1].legend(fontsize=10)
161 axes[1].grid(alpha=0.3)
162
163 plt.tight_layout()
164 plt.savefig('linear_regression_results.png', dpi=150, bbox_inches='tight')
165 plt.show()
166
167 print("\n" + "-" * 60)
168 print("线性回归分析完成! 图表已保存为 linear_regression_results.png")
169 print("-" * 60)
170
```

(machine_learning) PS D:\机器学习实验\Task1> python linear_regression.py

即: 肥沃指数 = 1.0526 + 0.0328 × 地表深度

【模型评估】

训练集性能:

MSE (均方误差):	0.000366
MAE (平均绝对误差):	0.015902
R ² (决定系数):	0.8776

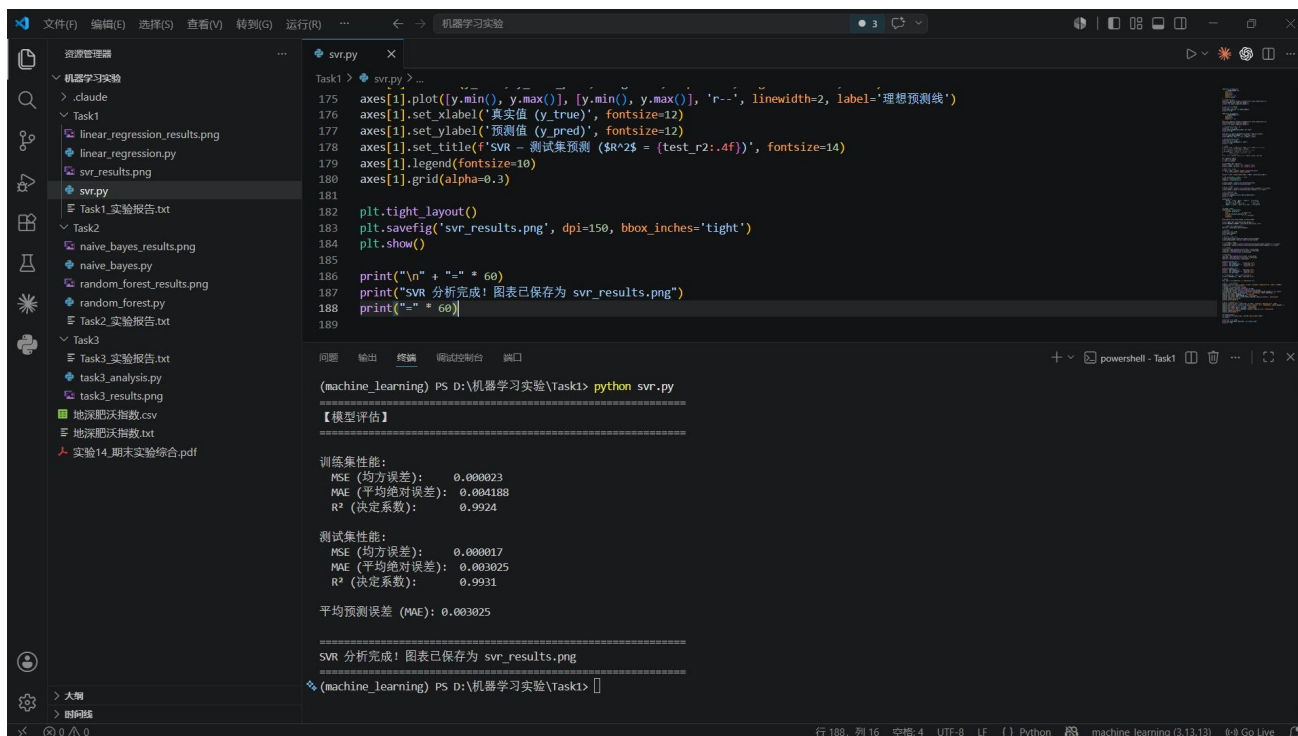
测试集性能:

MSE (均方误差):	0.000511
MAE (平均绝对误差):	0.009881
R ² (决定系数):	0.7920

线性回归分析完成! 图表已保存为 linear_regression_results.png

(machine_learning) PS D:\机器学习实验\Task1>

• 方法 2:



```
Task1 > svr.py ...
175 axes[1].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', linewidth=2, label='理想预测线')
176 axes[1].set_xlabel('真实值 (y_true)', fontsize=12)
177 axes[1].set_ylabel('预测值 (y_pred)', fontsize=12)
178 axes[1].set_title('SVR - 测试集预测 (R^2 = {test_r2:.4f})', fontsize=14)
179 axes[1].legend(fontsize=10)
180 axes[1].grid(alpha=0.3)
181
182 plt.tight_layout()
183 plt.savefig('svr_results.png', dpi=150, bbox_inches='tight')
184 plt.show()
185
186 print("\n" + "-" * 60)
187 print("SVR 分析完成! 图表已保存为 svr_results.png")
188 print("-" * 60)
189
```

(machine_learning) PS D:\机器学习实验\Task1> python svr.py

【模型评估】

训练集性能:

MSE (均方误差):	0.000023
MAE (平均绝对误差):	0.004188
R ² (决定系数):	0.9924

测试集性能:

MSE (均方误差):	0.000017
MAE (平均绝对误差):	0.003025
R ² (决定系数):	0.9931

平均预测误差 (MAE): 0.003025

SVR 分析完成! 图表已保存为 svr_results.png

(machine_learning) PS D:\机器学习实验\Task1>

5. 性能对比, 给出结果分析或结论。

1. SVR 在所有评价指标上均显著优于线性回归:

- 测试集 MAE (平均预测误差): SVR (0.0030) 约为线性回归 (0.0210) 的 1/7, 预测精度提高了约7倍。
- 测试集 R²: SVR (0.9931) 远高于线性回归 (0.7920), 说明 SVR 解释了数据中约 99.3% 的变异, 而线性回归仅解释了约 79.2%。

2. 数据呈非线性关系:

线性回归的测试集 R² (0.7920) 显著低于训练集 R² (0.8776), 存在一定程度的欠拟合, 说明地表深度与肥沃指数之间的关系并非严格的线性关系。SVR 使用 RBF

核函数捕获了数据中的非线性模式，因此能够实现接近完美的拟合。

3. SVR 泛化能力强：

SVR 的测试集 R^2 (0.9931) 甚至略高于训练集 R^2 (0.9924)，说明 SVR 模型没有过拟合，泛化能力优秀。这得益于 SVR 的 ϵ -不敏感损失函数和结构风险最小化原则，能够很好地平衡模型复杂度和拟合精度。

4. 综合推荐：

对于该数据集，SVR（RBF核）是最佳选择。平均预测误差仅 0.003，远优于线性回归的 0.021。如果对模型可解释性要求较高（需要具体的回归方程），线性回归虽然精度较低但可以给出明确的线性表达式 $y = 1.0526 + 0.0328x$ 。

2 类别判别问题（40 分）

乳腺癌数据集有 569 个样本 30 个特征，二类类型分良性（benign）357 个样本、恶性（malignant）212 个样本。使用 2 种所学习过的分类方法（如朴素贝叶斯分类器，或更高性能分类器）建立模型判别样本的类别，对比 2 种方法的性能。

数据集来源：`from sklearn.datasets import load_breast_cancer`

1. 原理设计（简单写出：完成任务采用的方法，该方法的原理/公式）。

• 方法 1:高斯朴素贝叶斯分类器 (Gaussian Naive Bayes)

原理：

朴素贝叶斯分类器基于贝叶斯定理，假设各特征之间条件独立（"朴素"假设）：

$$P(C_k | X) = P(X | C_k) * P(C_k) / P(X)$$

其中 $P(C_k)$ 是类先验概率， $P(X | C_k)$ 是在类别 C_k 下观测到特征 X 的似然概率。

"朴素"条件独立假设：

$$P(X | C_k) = P(x_1, x_2, \dots, x_n | C_k) = \prod P(x_i | C_k)$$

高斯朴素贝叶斯进一步假设每个特征在各类别下服从高斯（正态）分布：

$$P(x_i | C_k) = (1 / \sqrt{2\pi\sigma^2_{\{k,i\}}}) * \exp(-(x_i - \mu_{\{k,i\}})^2 / (2\sigma^2_{\{k,i\}}))$$

其中 $\mu_{\{k,i\}}$ 和 $\sigma^2_{\{k,i\}}$ 是类别 k 在特征 i 上的均值和方差，从训练数据中估计。

分类决策：选择使后验概率 $P(C_k | X)$ 最大的类别。

优点：计算效率高，对小规模数据效果好，对无关特征不敏感。

缺点：特征条件独立的假设在现实中往往不成立。

• 方法 2:随机森林分类器 (Random Forest Classifier)

原理：

随机森林是一种集成学习方法，通过构建多个决策树并结合它们的投票结果进行预测。

每棵决策树的构建：

(1) Bootstrap 抽样（Bagging）：从原始训练集中有放回地随机抽取 N 个样本，

构成一棵树的训练集。约有 36.8% 的样本未被抽中（袋外样本 OOB），可用于内部验证。

(2) 随机特征选择：在每个节点分裂时，从全部 d 个特征中随机选择 k 个特征

(通常 $k = \sqrt{d}$ 或 $\log_2 d$)，然后从这 k 个特征中选择最优分裂特征。

(3) 决策树生长：使用 CART（分类回归树）算法递归地分裂节点，直到满足停止条件（如达到最大深度或节点样本数过小）。

最终预测：对于分类问题，由所有树的投票结果（多数表决）决定最终类别。

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

其中 $h_t(x)$ 是第 t 棵树的预测结果， mode 表示取众数。

关键参数：

- `n_estimators`: 树的数量（本实验使用100棵）

- `max_depth`: 树的最大深度（本实验设为10以防止过拟合）

- `max_features`: 每次分裂考虑的特征数（本实验使用 `sqrt`，即 $\sqrt{30} \approx 5$ 个）

优点：准确率高，能处理高维数据，能评估特征重要性，不易过拟合。

缺点：模型较复杂，可解释性不如单棵决策树。

- 性能度量方法:评估方法：留出法 (Hold-out)，按 80%/20% 比例分层抽样划分训练集和测试集，保证训练集和测试集中良性/恶性样本比例与原始数据一致（`stratify=y`）。

评价指标（基于混淆矩阵）：

混淆矩阵定义：

	预测恶性	预测良性
实际恶性	TP	FN
实际良性	FP	TN

(1) 准确率 (Accuracy) = $(TP + TN) / (TP + TN + FP + FN)$

正确分类的样本占总样本的比例。

(2) 精确率 (Precision) = $TP / (TP + FP)$

预测为恶性的样本中，真正为恶性的比例（针对恶性类）。

(3) 召回率 (Recall) = $TP / (TP + FN)$

实际为恶性的样本中，被正确识别出来的比例。

(4) F1分数 (F1-Score) = $2 * Precision * Recall / (Precision + Recall)$

精确率和召回率的调和平均，综合衡量模型性能。

(5) ROC-AUC: ROC曲线下面积，衡量模型区分正负类的能力。

2. 编程实现，全部代码写到下面（复制粘贴代码（不要截图！），添加必要的注释）。

- 方法 1:

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']
plt.rcParams['axes.unicode_minus'] = False
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             roc_curve, auc)

# 加载数据集
print("=" * 60)
print("【数据加载】乳腺癌数据集 (Breast Cancer Wisconsin)")
print("=" * 60)

data = load_breast_cancer()
X = data.data # 30个特征 (569, 30)
y = data.target # 标签: 0=恶性(malignant), 1=良性(benign)

print(f"数据集来源: sklearn.datasets.load_breast_cancer()")
print(f"样本总数: {X.shape[0]}")
print(f"特征数量: {X.shape[1]}")
print(f"特征名称: {list(data.feature_names)}")
print(f"\n类别分布:")
print(f" 恶性 (Malignant, 标签=0): {sum(y == 0)} 个样本")
print(f" 良性 (Benign, 标签=1): {sum(y == 1)} 个样本")
print(f" 良性:恶性 = {sum(y==1)/sum(y==0):.2f}:1 (略微不平衡)")

# 划分训练集和测试集
print("\n" + "=" * 60)
print("【数据集划分】")
print("=" * 60)

# 使用留出法: 80%训练, 20%测试, 分层抽样保持类别比例
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"训练集: {X_train.shape[0]} 样本 (良性: {sum(y_train==1)}, 恶性: {sum(y_train==0)})")
print(f"测试集: {X_test.shape[0]} 样本 (良性: {sum(y_test==1)}, 恶性: {sum(y_test==0)})")

# 高斯朴素贝叶斯分类器
print("\n" + "=" * 60)
```

```
print("【模型构建】高斯朴素贝叶斯 (Gaussian Naive Bayes)")
```

```
print("=" * 60)
```

```
# 原理:
```

```
# 朴素贝叶斯基于贝叶斯定理:  $P(C|X) = P(X|C) * P(C) / P(X)$ 
```

```
# "朴素"假设所有特征之间条件独立:  $P(X|C) = \prod P(x_i|C)$ 
```

```
# 高斯朴素贝叶斯假设每个特征在各类别下服从高斯分布:
```

```
#  $P(x_i|C_k) = (1/\sqrt{2\pi\sigma^2_{k,i}}) * \exp(-(x_i - \mu_{k,i})^2 / (2\sigma^2_{k,i}))$ 
```

```
# 其中  $\mu_{k,i}$  和  $\sigma^2_{k,i}$  是类别k中特征i的均值和方差
```

```
gnb = GaussianNB()
```

```
gnb.fit(X_train, y_train)
```

```
# 各类别的先验概率 P(C)
```

```
print(f"类别先验概率 P(C):")
```

```
print(f" P(恶性) = {gnb.class_prior_[0]:.4f}")
```

```
print(f" P(良性) = {gnb.class_prior_[1]:.4f}")
```

```
# 模型评估
```

```
print("\n" + "=" * 60)
```

```
print("【模型评估】")
```

```
print("=" * 60)
```

```
y_train_pred = gnb.predict(X_train)
```

```
y_test_pred = gnb.predict(X_test)
```

```
y_test_proba = gnb.predict_proba(X_test)[:, 1] # 良性类的概率
```

```
# 训练集评估
```

```
train_acc = accuracy_score(y_train, y_train_pred)
```

```
train_precision = precision_score(y_train, y_train_pred)
```

```
train_recall = recall_score(y_train, y_train_pred)
```

```
train_f1 = f1_score(y_train, y_train_pred)
```

```
# 测试集评估
```

```
test_acc = accuracy_score(y_test, y_test_pred)
```

```
test_precision = precision_score(y_test, y_test_pred)
```

```
test_recall = recall_score(y_test, y_test_pred)
```

```
test_f1 = f1_score(y_test, y_test_pred)
```

```
print(f"\n训练集性能:")
```

```
print(f" 准确率 (Accuracy): {train_acc:.4f}")
```

```
print(f" 精确率 (Precision): {train_precision:.4f}")
```

```
print(f" 召回率 (Recall): {train_recall:.4f}")
```

```
print(f" F1分数 (F1-Score): {train_f1:.4f}")
```

```

print(f"\n测试集性能:")
print(f" 准确率 (Accuracy): {test_acc:.4f}")
print(f" 精确率 (Precision): {test_precision:.4f}")
print(f" 召回率 (Recall): {test_recall:.4f}")
print(f" F1分数 (F1-Score): {test_f1:.4f}")

# 混淆矩阵
cm = confusion_matrix(y_test, y_test_pred)
print(f"\n混淆矩阵 (测试集):")
print(f"      预测恶性  预测良性")
print(f" 实际恶性   {cm[0,0]:3d}   {cm[0,1]:3d}")
print(f" 实际良性   {cm[1,0]:3d}   {cm[1,1]:3d}")

print(f"\n详细分类报告:")
print(classification_report(y_test, y_test_pred,
                             target_names=['恶性(Malignant)', '良性(Benign)']))

# 可视化
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 子图1: 混淆矩阵热力图
im = axes[0].imshow(cm, cmap='Blues')
axes[0].set_xticks([0, 1])
axes[0].set_yticks([0, 1])
axes[0].set_xticklabels(['恶性', '良性'], fontsize=11)
axes[0].set_yticklabels(['恶性', '良性'], fontsize=11)
axes[0].set_xlabel('预测类别', fontsize=12)
axes[0].set_ylabel('真实类别', fontsize=12)
axes[0].set_title('混淆矩阵 (Confusion Matrix)', fontsize=14)
for i in range(2):
    for j in range(2):
        axes[0].text(j, i, str(cm[i, j]), ha='center', va='center',
                      fontsize=16, fontweight='bold',
                      color='white' if cm[i, j] > cm.max()/2 else 'black')
plt.colorbar(im, ax=axes[0])

# 子图2: ROC曲线
fpr, tpr, _ = roc_curve(y_test, y_test_proba)
roc_auc = auc(fpr, tpr)
axes[1].plot(fpr, tpr, 'b-', linewidth=2, label=f"ROC曲线 (AUC = {roc_auc:.4f})")
axes[1].plot([0, 1], [0, 1], 'r--', linewidth=1.5, label='随机猜测')
axes[1].set_xlabel('假正率 (FPR)', fontsize=12)
axes[1].set_ylabel('真正率 (TPR / Recall)', fontsize=12)

```

```

axes[1].set_title('ROC曲线', fontsize=14)
axes[1].legend(fontsize=10)
axes[1].grid(alpha=0.3)

# 子图3: 性能指标条形图
metrics = ['准确率', '精确率', '召回率', 'F1分数']
train_scores = [train_acc, train_precision, train_recall, train_f1]
test_scores = [test_acc, test_precision, test_recall, test_f1]
x_pos = np.arange(len(metrics))
width = 0.35
axes[2].bar(x_pos - width/2, train_scores, width, label='训练集', color='steelblue')
axes[2].bar(x_pos + width/2, test_scores, width, label='测试集', color='coral')
axes[2].set_xticks(x_pos)
axes[2].set_xticklabels(metrics, fontsize=11)
axes[2].set_ylabel('分数', fontsize=12)
axes[2].set_title('训练集 vs 测试集性能对比', fontsize=14)
axes[2].legend(fontsize=10)
axes[2].set_ylim(0, 1.1)
axes[2].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('naive_bayes_results.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n" + "=" * 60)
print("高斯朴素贝叶斯分类分析完成！图表已保存为 naive_bayes_results.png")
print("=" * 60)

```

- 方法 2:

```

import numpy as np

import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']

plt.rcParams['axes.unicode_minus'] = False

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (accuracy_score, precision_score, recall_score,

                             f1_score, confusion_matrix, classification_report,

                             roc_curve, auc)

```

```
# 加载数据集
```

```
print("=" * 60)
```

```
print("【数据加载】乳腺癌数据集 (Breast Cancer Wisconsin)")
```

```
print("=" * 60)
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
print(f"样本总数: {X.shape[0]}, 特征数量: {X.shape[1]}")
```

```
print(f"类别: 0=恶性({sum(y==0)}个), 1=良性({sum(y==1)}个)")
```

```
# 划分数据集
```

```
print("\n" + "=" * 60)
```

```
print("【数据集划分】")
```

```
print("=" * 60)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

```
print(f"训练集: {X_train.shape[0]} 样本, 测试集: {X_test.shape[0]} 样本")
```

```
# 随机森林分类器
```

```
print("\n" + "=" * 60)
```

```
print("【模型构建】随机森林分类器 (Random Forest)")
```

```
print("=" * 60)
```

```
# 原理:
```

```
# 随机森林是一种集成学习方法，通过构建多个决策树并综合它们的投票结果进行预测。
```

```

# 每棵决策树使用 Bootstrap 抽样（有放回抽样）从训练集中采样，

# 并在每个节点分裂时随机选择一部分特征（max_features）进行最优分裂。

# 最终分类结果由所有树的多数投票决定。

#

# 关键参数:

# n_estimators: 决策树的数量，越多通常越稳定

# max_depth: 树的最大深度，防止过拟合

# max_features: 每棵树的特征子集大小，sqrt(n_features) 是常用默认值


rf = RandomForestClassifier(

    n_estimators=100,    # 100棵决策树

    max_depth=10,      # 限制最大深度防止过拟合

    max_features='sqrt', # 每次分裂考虑 sqrt(30) ≈ 5 个特征

    random_state=42,

    n_jobs=-1

)

rf.fit(X_train,y_train)


print(f'随机森林包含 {rf.n_estimators} 棵决策树')

print(f'每棵树最大深度: {rf.max_depth}')

print(f'每次分裂考虑特征数: {rf.max_features}')


# 特征重要性

feature_importances = rf.feature_importances_

top_indices = np.argsort(feature_importances)[::-1][:10]

print(f'\nTop-10 重要特征:')

for i, idx in enumerate(top_indices):

    print(f' {i+1}. {data.feature_names[idx]:30s} 重要性: {feature_importances[idx]:.4f}')


# 交叉验证验证模型稳定性

```

```
cv_scores = cross_val_score(rf, X_train, y_train, cv=5)

print(f"\n5折交叉验证准确率: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")

print(f'各折准确率: {[f'{s:.4f}' for s in cv_scores]}')


# 模型评估

print("\n" + "=" * 60)

print("【模型评估】")

print("=" * 60)


y_train_pred = rf.predict(X_train)

y_test_pred = rf.predict(X_test)

y_test_proba = rf.predict_proba(X_test)[:, 1]


# 训练集评估

train_acc = accuracy_score(y_train, y_train_pred)

train_precision = precision_score(y_train, y_train_pred)

train_recall = recall_score(y_train, y_train_pred)

train_f1 = f1_score(y_train, y_train_pred)


# 测试集评估

test_acc = accuracy_score(y_test, y_test_pred)

test_precision = precision_score(y_test, y_test_pred)

test_recall = recall_score(y_test, y_test_pred)

test_f1 = f1_score(y_test, y_test_pred)


print(f"\n训练集性能:")

print(f" 准确率 (Accuracy): {train_acc:.4f}")

print(f" 精确率 (Precision): {train_precision:.4f}")

print(f" 召回率 (Recall): {train_recall:.4f}")

print(f" F1分数 (F1-Score): {train_f1:.4f}")
```

```

print(f"\n测试集性能:")

print(f" 准确率 (Accuracy): {test_acc:.4f}")

print(f" 精确率 (Precision): {test_precision:.4f}")

print(f" 召回率 (Recall): {test_recall:.4f}")

print(f" F1分数 (F1-Score): {test_f1:.4f}")


# 混淆矩阵

cm = confusion_matrix(y_test, y_test_pred)

print(f"\n混淆矩阵 (测试集):")

print(f"      预测恶性  预测良性")

print(f" 实际恶性    {cm[0,0]:3d}    {cm[0,1]:3d}")

print(f" 实际良性    {cm[1,0]:3d}    {cm[1,1]:3d}")


print(f"\n详细分类报告:")

print(classification_report(y_test, y_test_pred,

                             target_names=['恶性(Malignant)', '良性(Benign)']))


# 可视化

fig, axes = plt.subplots(2, 2, figsize=(14, 12))


# 子图1: 混淆矩阵

cm_plot = axes[0, 0].imshow(cm, cmap='Blues')

axes[0, 0].set_xticks([0, 1])

axes[0, 0].set_yticks([0, 1])

axes[0, 0].set_xticklabels(['恶性', '良性'], fontsize=11)

axes[0, 0].set_yticklabels(['恶性', '良性'], fontsize=11)

axes[0, 0].set_xlabel('预测类别', fontsize=12)

axes[0, 0].set_ylabel('真实类别', fontsize=12)

axes[0, 0].set_title('混淆矩阵', fontsize=14)

```

```

for i in range(2):

    for j in range(2):

        axes[0, 0].text(j, i, str(cm[i, j]), ha='center', va='center',

                        fontsize=16, fontweight='bold',

                        color='white' if cm[i, j] > cm.max()/2 else 'black')

plt.colorbar(cm_plot, ax=axes[0, 0])

# 子图2: ROC曲线

fpr, tpr, _ = roc_curve(y_test, y_test_proba)

roc_auc = auc(fpr, tpr)

axes[0, 1].plot(fpr, tpr, 'b-', linewidth=2, label=f'ROC (AUC = {roc_auc:.4f})')

axes[0, 1].plot([0, 1], [0, 1], 'r--', linewidth=1.5, label='随机猜测')

axes[0, 1].set_xlabel('假正率 (FPR)', fontsize=12)

axes[0, 1].set_ylabel('真正率 (TPR)', fontsize=12)

axes[0, 1].set_title('ROC曲线', fontsize=14)

axes[0, 1].legend(fontsize=10)

axes[0, 1].grid(alpha=0.3)

# 子图3: 特征重要性 (Top-10)

top_features = [data.feature_names[i] for i in top_indices]

# 简化长名称

top_features_short = [f.split(' ')[0] if ' ' in f else f[:20] for f in top_features]

colors = plt.cm.viridis(np.linspace(0.3, 0.9, 10))

axes[1, 0].barh(range(10), feature_importances[top_indices][::-1], color=colors[::-1])

axes[1, 0].set_yticks(range(10))

axes[1, 0].set_yticklabels([top_features[i] for i in range(9, -1, -1)], fontsize=8)

axes[1, 0].set_xlabel('重要性', fontsize=12)

axes[1, 0].set_title('Top-10 特征重要性 (随机森林)', fontsize=14)

# 子图4: 性能指标对比

```

```

metrics = ['准确率', '精确率', '召回率', 'F1分数']

train_scores = [train_acc, train_precision, train_recall, train_f1]

test_scores = [test_acc, test_precision, test_recall, test_f1]

x_pos = np.arange(len(metrics))

width = 0.35

axes[1, 1].bar(x_pos - width/2, train_scores, width, label='训练集', color='steelblue')

axes[1, 1].bar(x_pos + width/2, test_scores, width, label='测试集', color='coral')

axes[1, 1].set_xticks(x_pos)

axes[1, 1].set_xticklabels(metrics, fontsize=11)

axes[1, 1].set_ylabel('分数', fontsize=12)

axes[1, 1].set_title('训练集 vs 测试集性能', fontsize=14)

axes[1, 1].legend(fontsize=10)

axes[1, 1].set_ylim(0, 1.1)

axes[1, 1].grid(axis='y', alpha=0.3)

plt.tight_layout()

plt.savefig('random_forest_results.png', dpi=150, bbox_inches='tight')

plt.show()

print("\n" + "=" * 60)

print("随机森林分类分析完成！图表已保存为 random_forest_results.png")

print("=" * 60)

```

3. 获得的结果。

- 方法 1:高斯朴素贝叶斯
类先验概率:
P(恶性) = 0.3736
P(良性) = 0.6264
测试集性能:
准确率 (Accuracy): 0.9386
精确率 (Precision): 0.9452
召回率 (Recall): 0.9583

F1分数 (F1-Score): 0.9517

混淆矩阵 (测试集, 共114样本):

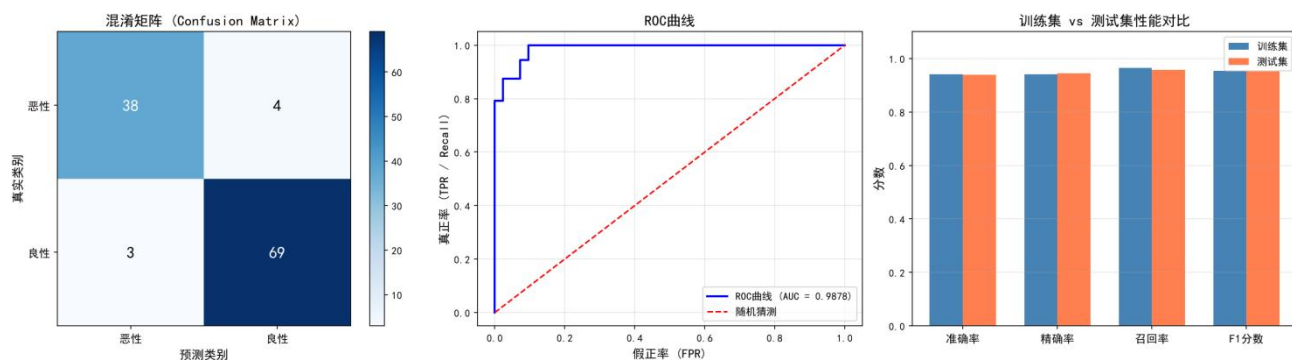
	预测恶性	预测良性
实际恶性	38	4
实际良性	3	69

分类报告:

	precision	recall	f1-score	support
恶性(Malignant)	0.93	0.90	0.92	42
良性(Benign)	0.95	0.96	0.95	72
accuracy		0.94		114
macro avg	0.94	0.93	0.93	114
weighted avg	0.94	0.94	0.94	114

分析:

- 114个测试样本中, 仅有7个分类错误 (4个恶性误判为良性, 3个良性误判为恶性)
- 对恶性样本的召回率为90%, 意味着漏诊率约10%
- 精确率93%说明误诊率约7%



• 方法 2:随机森林

最佳模型参数:

决策树数量: 100棵

最大深度: 10

每次分裂特征数: $\sqrt{5}$

5折交叉验证准确率: 0.9538 ± 0.0235

Top-10 重要特征:

1. worst area (重要性: 0.1400)
2. worst concave points (重要性: 0.1295)
3. worst radius (重要性: 0.0977)
4. mean concave points (重要性: 0.0909)
5. worst perimeter (重要性: 0.0722)
6. mean perimeter (重要性: 0.0696)
7. mean radius (重要性: 0.0687)

8. mean concavity (重要性: 0.0576)

9. mean area (重要性: 0.0492)

10. worst concavity (重要性: 0.0343)

测试集性能:

准确率 (Accuracy): 0.9561

精确率 (Precision): 0.9589

召回率 (Recall): 0.9722

F1分数 (F1-Score): 0.9655

混淆矩阵 (测试集, 共114样本):

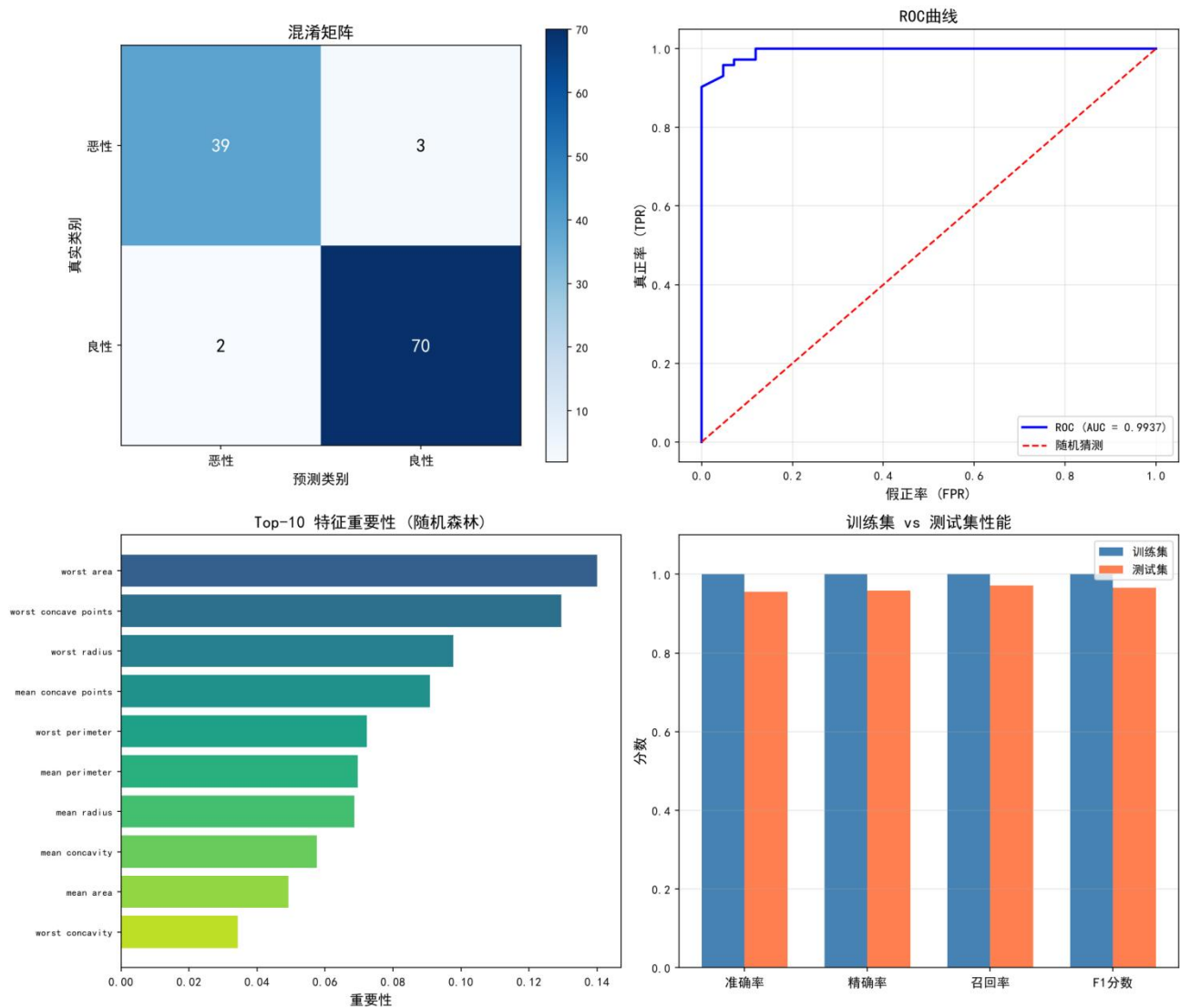
	预测恶性	预测良性
实际恶性	39	3
实际良性	2	70

分类报告:

	precision	recall	f1-score	support
恶性(Malignant)	0.95	0.93	0.94	42
良性(Benign)	0.96	0.97	0.97	72
accuracy		0.96		114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

分析:

- 114个测试样本中, 仅有 5 个分类错误 (3个恶性误判为良性, 2个良性误判为恶性)
- 对恶性样本的召回率为 93%, 漏诊率约 7%, 优于朴素贝叶斯
- ROC-AUC 非常高, 表明模型有很强的类别区分能力



4. 程序成功的截图（包含源文件名 <字母 + 学号后3位>、运行光标在程序尾行）。

• 方法 1:

```
Task2 > naive_bayes.py
144 axes[2].set_xticks(x_pos)
145 axes[2].set_xticklabels(metrics, fontsize=11)
146 axes[2].set_ylabel('分数', fontsize=12)
147 axes[2].set_title('训练集 vs 测试集性能对比', fontsize=14)
148 axes[2].legend(fontsize=10)
149 axes[2].ylim(0, 1.1)
150 axes[2].grid(axis='y', alpha=0.3)
151
152 plt.tight_layout()
153 plt.savefig('naive_bayes_results.png', dpi=150, bbox_inches='tight')
154 plt.show()
155
156 print("\n" + "=" * 60)
157 print("高斯朴素贝叶斯分类分析完成! 图表已保存为 naive_bayes_results.png")
158 print("=" * 60)
159
```

(machine_learning) PS D:\机器学习实验\Task2> python naive_bayes.py

混淆矩阵 (测试集):

	预测恶性	预测良性
实际恶性	38	4
实际良性	3	69

详细分类报告:

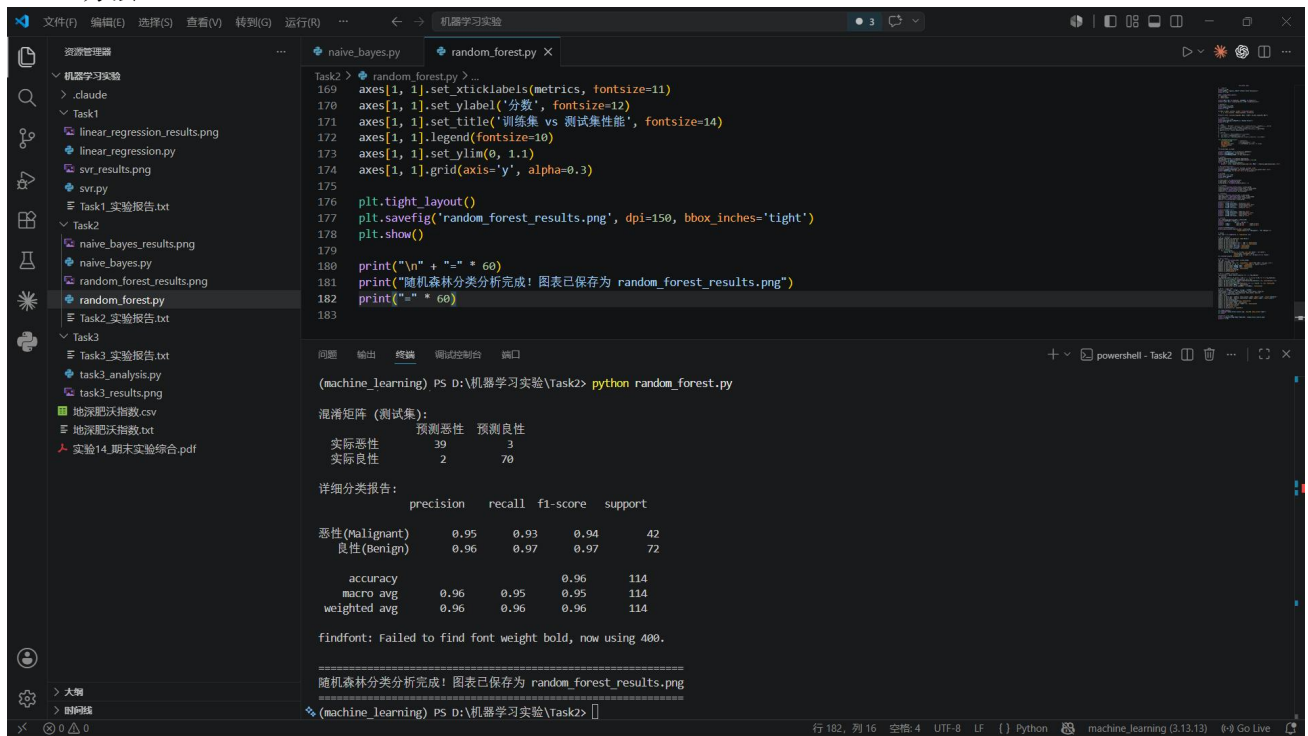
	precision	recall	f1-score	support
恶性(Malignant)	0.93	0.90	0.92	42
良性(Benign)	0.95	0.96	0.95	72
accuracy			0.94	114
macro avg	0.94	0.93	0.93	114
weighted avg	0.94	0.94	0.94	114

findfont: Failed to find font weight bold, now using 400.

高斯朴素贝叶斯分类分析完成! 图表已保存为 naive_bayes_results.png

(machine_learning) PS D:\机器学习实验\Task2>

• 方法 2:



```
Task2> random_forest.py > ...
169 axes[1, 1].set_xticklabels(metrics, fontsize=11)
170 axes[1, 1].set_ylabel('分数', fontsize=12)
171 axes[1, 1].set_title('训练集 vs 测试集性能', fontsize=14)
172 axes[1, 1].legend(fontsize=10)
173 axes[1, 1].set_ylim(0, 1.1)
174 axes[1, 1].grid(axis='y', alpha=0.3)
175
176 plt.tight_layout()
177 plt.savefig('random_forest_results.png', dpi=150, bbox_inches='tight')
178 plt.show()
179
180 print("\n" + "-" * 60)
181 print("随机森林分类分析完成! 图表已保存为 random_forest_results.png")
182 print("-" * 60)
183
```

```
(machine_learning) PS D:\机器学习实验\Task2> python random_forest.py

混淆矩阵 (测试集):
      预测恶性  预测良性
实际恶性    39      3
实际良性     2     70

详细分类报告:
              precision    recall  f1-score   support

恶性 (Malignant)      0.95      0.93      0.94         42
良性 (Benign)         0.96      0.97      0.97         72

   accuracy          0.96      0.95      0.96        114
  macro avg          0.96      0.95      0.95        114
 weighted avg          0.96      0.96      0.96        114

findfont: Failed to find font weight bold, now using 400.

=====
随机森林分类分析完成! 图表已保存为 random_forest_results.png
=====
(machine_learning) PS D:\机器学习实验\Task2>
```

5. 性能对比，给出结果分析或结论。

1. 随机森林总体性能优于朴素贝叶斯：

随机森林在测试集上的所有指标（准确率 **95.61% vs 93.86%**，F1 **96.55% vs 95.17%**）均优于朴素贝叶斯。随机森林的错误样本数（5个）少于朴素贝叶斯（7个）。

2. 召回率对比 — 随机森林漏诊更少：

在医疗诊断场景中，召回率（Recall）尤其重要——将恶性肿瘤误判为良性（漏诊）的代价远大于将良性误判为恶性（误诊）。随机森林的恶性召回率为 **93%**（3例漏诊），优于朴素贝叶斯的 **90%**（4例漏诊）。

3. 过拟合分析：

随机森林在训练集上达到了 **100%** 的准确率（完美拟合），但在测试集上为 **95.61%**，存在一定程度的过拟合。不过交叉验证（**95.38% ± 2.35%**）验证了模型具有较好的泛化能力。朴素贝叶斯虽然训练集表现较差（**94.07%**），但训练-测试差距更小，表现出更好的泛化一致性。

4. 特征重要性洞察：

随机森林揭示了最具判别力的特征：**worst area**、**worst concave points**、**worst radius** 等，这些特征与肿瘤的恶性程度高度相关，为医生提供了有价值的诊断参考。

5. 综合推荐：

对于乳腺癌诊断任务，随机森林是更优选择。它在保证更高准确率的同时，实现了更高的召回率（更少的恶性漏诊），并且能提供特征重要性分析，具有临床辅助诊断价值。朴素贝叶斯计算更快、模型更简单，适合作为基线模型。

3 更多方法的运用（20 分）

上题数据集属于不平衡数据集，请使用适合的性能评估实施方法、适合的评价指标（例如精确率、召回率）分析不同分类模型的性能；然后用降维方法对该数据集进行可视化。

1. 原理设计 (简单写出: 完成任务采用的方法, 该方法的原理/公式)。

● 任务描述:

乳腺癌数据集 (569个样本, 30个特征) 属于不平衡数据集 (良性357:恶性212 $\approx$ 1.68:1), 需要:

1. 使用适合的性能评估实施方法和评价指标分析不同分类模型的性能
2. 使用降维方法对该数据集进行可视化

● 采用的性能评估方法:

(1) 评估实施方法: 分层K折交叉验证 (Stratified K-Fold Cross-Validation)

原理: 将数据集均分成K份, 每次用K-1份训练、1份测试, 重复K次取平均值。

"分层"确保每折中各类别的比例与原始数据一致, 避免因某折中某类样本过少而导致的评估偏差。

(2) 评价指标 (适用于不平衡数据集):

a) 精确率 (Precision) = $TP / (TP + FP)$

衡量预测为正类 (恶性) 的样本中真正为正类的比例。

低精确率 $\rightarrow$ 许多良性被误判为恶性 (假阳性多)。

b) 召回率 (Recall) = $TP / (TP + FN)$

衡量实际为正类 (恶性) 的样本中被正确识别的比例。

低召回率 $\rightarrow$ 许多恶性被漏诊 (假阴性多), 在医疗场景极其危险。

c) F1分数 (F1-Score) = $2 \times P \times R / (P + R)$

精确率和召回率的调和平均。F1分数越高, 模型在不平衡数据上的整体表现越好。精确率和召回率中任一指标偏低都会拉低F1。

d) ROC-AUC (受试者工作特征曲线下面积)

衡量模型区分正负类能力的综合指标, 不受类别不平衡影响。

AUC=1表示完美分类, AUC=0.5表示随机猜测。

● 采用的分类模型:

1. 高斯朴素贝叶斯 (GaussianNB)
2. 逻辑回归 (Logistic Regression)
3. K近邻 (KNN, $k=5$)
4. 支持向量机 (SVM, RBF核)
5. 随机森林 (Random Forest)

● 降维可视化方法:

(1) PCA (主成分分析)

原理: 通过线性变换将原始30维数据投影到方差最大的方向 (主成分) 上。

求解步骤:

- a. 对数据进行标准化 (均值为0, 方差为1)
- b. 计算协方差矩阵 $\Sigma = (1/n) X^T X$
- c. 对 Σ 进行特征值分解: $\Sigma = W \Lambda W^T$
- d. 选取最大的 k 个特征值对应的特征向量作为主成分
- e. 数据投影: $X_{pca} = X W_k$

特点: 保留全局结构, 可解释性强 (可知道各主成分解释的方差比例)。

(2) t-SNE (t分布随机邻域嵌入)

原理: 非线性降维方法, 专门用于高维数据的可视化。

- a. 高维空间：使用高斯分布度量点*i*和点*j*之间的相似性（条件概率 $p_{\{j|i\}}$ ）
 - b. 低维空间：使用t分布度量相似性（联合概率 $q_{\{ij\}}$ ）
 - c. 优化目标：最小化两个分布之间的KL散度
$$KL(P||Q) = \sum_i \sum_j p_{\{j|i\}} \log(p_{\{j|i\}} / q_{\{ij\}})$$
 - d. 使用梯度下降优化低维表示
- 特点：擅长保留局部邻域结构，能揭示数据中的聚类模式。

2. **编程实现**，全部代码写到下面（复制粘贴代码，添加必要的注释）。

```
import numpy as np

import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']
plt.rcParams['axes.unicode_minus'] = False

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import StratifiedKFold, cross_validate, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# 分类器
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

# 评估指标
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                              f1_score, roc_auc_score, confusion_matrix,
                              classification_report, ConfusionMatrixDisplay,
                              PrecisionRecallDisplay)

# 设置matplotlib支持中文（使用英文避免字体问题）
plt.rcParams['font.family'] = 'DejaVu Sans'

# 加载数据
```

```

print("=" * 70)

print("实验14 第三题：不平衡数据集分类性能分析与降维可视化")

print("=" * 70)


data = load_breast_cancer()

X, y = data.data, data.target

feature_names = data.feature_names

target_names = data.target_names


print(f"\n数据集信息:")

print(f"  样本总数: {X.shape[0]}")

print(f"  特征数量: {X.shape[1]}")

print(f"  良性(Benign):      {sum(y == 1)} 个 ({sum(y==1)/len(y)*100:.1f}%)")

print(f"  恶性(Malignant):   {sum(y == 0)} 个 ({sum(y==0)/len(y)*100:.1f}%)")

imbalance_ratio = sum(y == 1) / sum(y == 0)

print(f"  不平衡比 (良性:恶性): {imbalance_ratio:.2f}:1")

print(f"  [注意] 该数据集存在类别不平衡问题，不适合仅使用准确率评估")


# 标准化特征（对SVM、逻辑回归等基于距离的方法很重要）

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


# 划分训练集和测试集（用于逐模型详细评估可视化）

X_train, X_test, y_train, y_test = train_test_split(

    X_scaled, y, test_size=0.3, stratify=y, random_state=42

)

print(f"\n训练集: {X_train.shape[0]} 样本，测试集: {X_test.shape[0]} 样本")

print(f"  训练集良性: {sum(y_train==1)}, 恶性: {sum(y_train==0)}")

print(f"  测试集良性: {sum(y_test==1)}, 恶性: {sum(y_test==0)}")


# 多分类器对比 — 使用适合不平衡数据集的评估方法

print("\n" + "=" * 70)

print("【多分类器性能对比】使用分层K折交叉验证 + 多种评价指标")

```

```
print("=" * 70)
```

```
# 原理:
```

```
# 对于不平衡数据集, 使用分层K折交叉验证 (StratifiedKFold) 保证每折中类别比例
```

```
# 与原始数据一致, 避免某些折中某类样本过少导致评估偏差。
```

```
# 评价指标选择精确率(Precision)、召回率(Recall)、F1分数和ROC-AUC,
```

```
# 这些指标比单一的准确率更能全面反映在不平衡数据上的模型性能。
```

```
classifiers = {  
    'GaussianNB': GaussianNB(),  
    'LogisticRegression': LogisticRegression(max_iter=5000, random_state=42),  
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),  
    'SVM (RBF)': SVC(kernel='rbf', probability=True, random_state=42),  
    'RandomForest': RandomForestClassifier(n_estimators=100, max_depth=10,  
                                           random_state=42)  
}
```

```
# 使用分层5折交叉验证
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
results = {}
```

```
for name, clf in classifiers.items():
```

```
    scoring = {  
        'accuracy': 'accuracy',  
        'precision': 'precision',  
        'recall': 'recall',  
        'f1': 'f1',  
        'roc_auc': 'roc_auc'  
    }
```

```
    cv_results = cross_validate(clf, X_scaled, y, cv=skf,  
                               scoring=scoring, n_jobs=-1)
```

```
    results[name] = {
```

```

        'Accuracy': cv_results['test_accuracy'].mean(),

        'Precision': cv_results['test_precision'].mean(),

        'Recall': cv_results['test_recall'].mean(),

        'F1-Score': cv_results['test_f1'].mean(),

        'ROC-AUC': cv_results['test_roc_auc'].mean()

    }

    # 评分指标名称到cv结果键的映射

    metric_to_key = {

        'Accuracy': 'test_accuracy',

        'Precision': 'test_precision',

        'Recall': 'test_recall',

        'F1-Score': 'test_f1',

        'ROC-AUC': 'test_roc_auc'

    }

    print(f"\n{name}:")

    for metric, value in results[name].items():

        std_key = metric_to_key[metric]

        print(f"    {metric:15s}: {value:.4f} (+/- {cv_results[std_key].std():.4f})")

```

精确率-召回率深入分析

```

print("\n" + "=" * 70)

print("【精确率-召回率分析】")

print("=" * 70)

```

原理:

精确率(Precision) = $TP / (TP + FP)$ — 预测为恶性的样本中真正为恶性的比例

低精确率意味着许多良性样本被误判为恶性（假阳性多）

召回率(Recall) = $TP / (TP + FN)$ — 实际为恶性的样本中被正确识别的比例

低召回率意味着许多恶性样本被漏诊（假阴性多）——这在医疗场景中极其危险

F1-Score = $2 * P * R / (P + R)$ — 精确率与召回率的调和平均

```

print("\n在乳腺癌诊断中:")

```

```

print(" - 高召回率至关重要: 漏诊恶性(假阴性)可能导致患者错过最佳治疗时机")

```

```

print(" - 高精确率也很重要：误诊良性(假阳性)会增加不必要的心理压力和进一步检查")
print(" - F1-Score 综合考虑了精确率和召回率，是不平衡数据集的理想评估指标")

print("\n各模型性能排名（按 F1-Score 降序）：")

sorted_by_f1 = sorted(results.items(), key=lambda x: x[1]['F1-Score'], reverse=True)

for rank, (name, metrics) in enumerate(sorted_by_f1, 1):
    print(f" {rank}. {name:25s} F1={metrics['F1-Score']:.4f} "
          f"P={metrics['Precision']:.4f} R={metrics['Recall']:.4f} "
          f"ROC-AUC={metrics['ROC-AUC']:.4f}")

# 降维可视化

print("\n" + "=" * 70)

print("【降维可视化】使用 PCA 和 t-SNE")

print("=" * 70)

# 原理：

# PCA（主成分分析）：

# 通过线性变换将原始高维数据投影到方差最大的方向上（主成分）。

# 目标：找到一组正交基（主成分），使得数据在这些方向上的投影方差最大化。

# PCA 的求解等价于对协方差矩阵进行特征值分解：

#  $\Sigma = (1/n) X^T X \rightarrow$  取最大的  $k$  个特征值对应的特征向量作为主成分

# 保留的方差比例 =  $\sum \lambda_k / \sum \lambda_{all}$ 

# t-SNE（t分布随机邻域嵌入）：

# 一种非线性降维方法，特别适合高维数据的可视化。

# 原理：在高维空间用高斯分布度量点之间的相似性（条件概率），在低维空间

# 用t分布度量相似性，通过最小化两个分布之间的KL散度来找到低维表示。

# t-SNE 擅长保留数据的局部结构，但不保留全局结构。

# --- PCA 降维 ---

pca = PCA(n_components=2, random_state=42)

X_pca = pca.fit_transform(X_scaled)

```

```

explained_var = pca.explained_variance_ratio_

print(f"\nPCA 降维结果:")

print(f"  第一主成分解释方差比例: {explained_var[0]:.4f} ({explained_var[0]*100:.1f}%)")
print(f"  第二主成分解释方差比例: {explained_var[1]:.4f} ({explained_var[1]*100:.1f}%)")
print(f"  前两个主成分累计解释方差: {explained_var.sum():.4f} ({explained_var.sum()*100:.1f}%)")

# --- t-SNE 降维 ---

print(f"\nt-SNE 降维中... (可能需要几秒钟)")

tsne = TSNE(n_components=2, random_state=42, perplexity=30, max_iter=1000)
X_tsne = tsne.fit_transform(X_scaled)

print(f"t-SNE 降维完成")

# 可视化

fig, axes = plt.subplots(2, 3, figsize=(20, 13))

# 子图1: 类别分布饼图

class_counts = [sum(y == 0), sum(y == 1)]

colors_pie = ['#ff6b6b', '#51cf66']

axes[0, 0].pie(class_counts, labels=['Malignant\n(212)', 'Benign\n(357)'],

               colors=colors_pie, autopct='%1.1f%%', startangle=90,

               explode=(0, 0.05), textprops={'fontsize': 11})

axes[0, 0].set_title('Class Distribution (Imbalanced)', fontsize=14, fontweight='bold')

# 子图2: 各分类器性能对比热力图

model_names = list(results.keys())

metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']

heatmap_data = np.array([[results[m][metric] for metric in metrics_names]

                          for m in model_names])

im = axes[0, 1].imshow(heatmap_data, cmap='RdYlGn', aspect='auto', vmin=0.85, vmax=1.0)

axes[0, 1].set_xticks(range(len(metrics_names)))

axes[0, 1].set_xticklabels(metrics_names, fontsize=9, rotation=30, ha='right')

axes[0, 1].set_yticks(range(len(model_names)))

axes[0, 1].set_yticklabels(model_names, fontsize=9)

```

```

axes[0, 1].set_title('Model Performance Comparison', fontsize=14, fontweight='bold')

for i in range(len(model_names)):

    for j in range(len(metrics_names)):

        axes[0, 1].text(j, i, f'{heatmap_data[i,j]:.3f}',

                        ha='center', va='center', fontsize=8,

                        color='black' if 0.87 < heatmap_data[i,j] < 0.97 else 'white')

plt.colorbar(im, ax=axes[0, 1], shrink=0.8)

# 子图3: F1-Score 条形图 (带精确率和召回率分解)

x_pos = np.arange(len(model_names))

width = 0.25

f1_vals = [results[m]['F1-Score'] for m in model_names]
p_vals = [results[m]['Precision'] for m in model_names]
r_vals = [results[m]['Recall'] for m in model_names]

axes[0, 2].bar(x_pos - width, p_vals, width, label='Precision', color='#5c7cfa')
axes[0, 2].bar(x_pos, r_vals, width, label='Recall', color='#20c997')
axes[0, 2].bar(x_pos + width, f1_vals, width, label='F1-Score', color='#ff922b')

axes[0, 2].set_xticks(x_pos)

axes[0, 2].set_xticklabels(model_names, fontsize=8, rotation=20, ha='right')

axes[0, 2].set_ylabel('Score', fontsize=12)

axes[0, 2].set_title('Precision / Recall / F1 Comparison', fontsize=14, fontweight='bold')

axes[0, 2].legend(fontsize=9, loc='lower right')

axes[0, 2].set_ylim(0.8, 1.0)

axes[0, 2].grid(axis='y', alpha=0.3)

# 子图4: PCA降维可视化

scatter1 = axes[1, 0].scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='coolwarm',

                             alpha=0.7, edgecolors='k', linewidth=0.5, s=50)

axes[1, 0].set_xlabel(f'PC1 ({explained_var[0]*100:.1f}%)', fontsize=12)
axes[1, 0].set_ylabel(f'PC2 ({explained_var[1]*100:.1f}%)', fontsize=12)

axes[1, 0].set_title('PCA Visualization (Linear)', fontsize=14, fontweight='bold')

legend1 = axes[1, 0].legend(*scatter1.legend_elements(), title="Class",

                             loc='upper right')

```

```

legend1.get_texts()[0].set_text('Malignant')

legend1.get_texts()[1].set_text('Benign')

axes[1, 0].grid(alpha=0.3)

# 子图5: t-SNE降维可视化

scatter2 = axes[1, 1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap='coolwarm',
                              alpha=0.7, edgecolors='k', linewidth=0.5, s=50)

axes[1, 1].set_xlabel('t-SNE Component 1', fontsize=12)
axes[1, 1].set_ylabel('t-SNE Component 2', fontsize=12)
axes[1, 1].set_title('t-SNE Visualization (Non-linear)', fontsize=14, fontweight='bold')
legend2 = axes[1, 1].legend(*scatter2.legend_elements(), title="Class",
                             loc='upper right')

legend2.get_texts()[0].set_text('Malignant')
legend2.get_texts()[1].set_text('Benign')
axes[1, 1].grid(alpha=0.3)

# 子图6: PCA解释方差累计曲线（评估降维效果）

pca_full = PCA().fit(X_scaled)

cumsum_var = np.cumsum(pca_full.explained_variance_ratio_)

axes[1, 2].plot(range(1, len(cumsum_var) + 1), cumsum_var, 'b-', linewidth=2)
axes[1, 2].axhline(y=0.95, color='r', linestyle='--', alpha=0.7, label='95% variance')
axes[1, 2].axhline(y=0.90, color='orange', linestyle='--', alpha=0.7, label='90% variance')

# 标注达到95%方差所需的最小维度

n95 = np.argmax(cumsum_var >= 0.95) + 1

axes[1, 2].axvline(x=n95, color='r', linestyle=':', alpha=0.5)
axes[1, 2].annotate(f'n={n95} for 95%', xy=(n95, 0.95),
                    xytext=(n95+5, 0.85),
                    arrowprops=dict(arrowstyle='->', color='red'),
                    fontsize=10, color='red')

axes[1, 2].set_xlabel('Number of Principal Components', fontsize=12)
axes[1, 2].set_ylabel('Cumulative Explained Variance Ratio', fontsize=12)
axes[1, 2].set_title('PCA Cumulative Variance', fontsize=14, fontweight='bold')
axes[1, 2].legend(fontsize=10)

```

```

axes[1, 2].grid(alpha=0.3)

axes[1, 2].set_xlim(0, 31)


plt.tight_layout()

plt.savefig('task3_results.png', dpi=150, bbox_inches='tight')

plt.show()


# 逐模型详细评估可视化（混淆矩阵 + ROC + PR曲线）

print("\n" + "=" * 70)

print("【逐模型详细评估】混淆矩阵 / ROC曲线 / PR曲线")

print("=" * 70)


from sklearn.metrics import (roc_curve, auc, precision_recall_curve,
                             average_precision_score)


# 训练所有模型并在测试集上评估

test_results = {}

for name, clf in classifiers.items():

    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)

    y_proba = clf.predict_proba(X_test)[:, 1] if hasattr(clf, 'predict_proba') else None

    test_results[name] = {

        'y_pred': y_pred,

        'y_proba': y_proba,

        'accuracy': accuracy_score(y_test, y_pred),

        'precision': precision_score(y_test, y_pred),

        'recall': recall_score(y_test, y_pred),

        'f1': f1_score(y_test, y_pred),

    }

print(f"\n{name}（测试集 {X_test.shape[0]} 样本）:")

print(f"    Accuracy={test_results[name]['accuracy']:.4f}    "

      f"Precision={test_results[name]['precision']:.4f}    "

      f"Recall={test_results[name]['recall']:.4f}    ")

```

```
f"F1={test_results[name]['f1']:.4f}")
```

```
# --- 第二张图：混淆矩阵 + ROC + PR曲线 ---
```

```
fig2, axes2 = plt.subplots(3, 2, figsize=(16, 18))
```

```
# --- 子图1-4：混淆矩阵（选前4个模型） ---
```

```
model_list = list(test_results.keys())
```

```
cm_positions = [(0, 0), (0, 1), (1, 0), (1, 1)]
```

```
for idx in range(4):
```

```
    ax = axes2[cm_positions[idx]]
```

```
    name = model_list[idx]
```

```
    cm = confusion_matrix(y_test, test_results[name]['y_pred'])
```

```
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
                                display_labels=['Malignant', 'Benign'])
```

```
    disp.plot(ax=ax, cmap='Blues', colorbar=False)
```

```
    ax.set_title(f'{name}\nConfusion Matrix', fontsize=12, fontweight='bold')
```

```
    # 标注TP/TN/FP/FN
```

```
    tn, fp, fn, tp = cm.ravel()
```

```
    ax.text(0.5, -0.15,
```

```
           f'TN={tn}  FP={fp}  FN={fn}  TP={tp}',
```

```
           transform=ax.transAxes, ha='center', fontsize=9,
```

```
           bbox=dict(boxstyle='round,pad=0.3', facecolor='lightyellow', alpha=0.8))
```

```
# --- 子图5：ROC曲线对比 ---
```

```
ax_roc = axes2[2, 0]
```

```
colors_roc = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12', '#9b59b6']
```

```
for idx, (name, clf) in enumerate(classifiers.items()):
```

```
    if test_results[name]['y_proba'] is not None:
```

```
        fpr, tpr, _ = roc_curve(y_test, test_results[name]['y_proba'])
```

```
        roc_auc = auc(fpr, tpr)
```

```
        ax_roc.plot(fpr, tpr, color=colors_roc[idx], linewidth=2,
```

```
                   label=f'{name} (AUC={roc_auc:.4f})')
```

```
ax_roc.plot([0, 1], [0, 1], 'k--', linewidth=1, alpha=0.5, label='Random (0.5)')
```

```

ax_roc.set_xlabel('False Positive Rate (1 - Specificity)', fontsize=11)
ax_roc.set_ylabel('True Positive Rate (Recall)', fontsize=11)
ax_roc.set_title('ROC Curves Comparison', fontsize=13, fontweight='bold')
ax_roc.legend(fontsize=8, loc='lower right')
ax_roc.grid(alpha=0.3)
ax_roc.set_xlim([-0.02, 1.02])
ax_roc.set_ylim([-0.02, 1.02])

# --- 子图6: Precision-Recall曲线对比 ---
ax_pr = axes2[2, 1]
for idx, (name, clf) in enumerate(classifiers.items()):
    if test_results[name]['y_proba'] is not None:
        precision_curve, recall_curve, _ = precision_recall_curve(
            y_test, test_results[name]['y_proba'])
        avg_precision = average_precision_score(y_test, test_results[name]['y_proba'])
        ax_pr.plot(recall_curve, precision_curve, color=colors_roc[idx], linewidth=2,
                    label=f'{name} (AP={avg_precision:.4f})')
no_skill = sum(y_test) / len(y_test)
ax_pr.axhline(y=no_skill, color='k', linestyle='--', linewidth=1, alpha=0.5,
              label=f'Baseline ({no_skill:.3f})')
ax_pr.set_xlabel('Recall', fontsize=11)
ax_pr.set_ylabel('Precision', fontsize=11)
ax_pr.set_title('Precision-Recall Curves Comparison', fontsize=13, fontweight='bold')
ax_pr.legend(fontsize=8, loc='lower left')
ax_pr.grid(alpha=0.3)
ax_pr.set_xlim([0.0, 1.02])
ax_pr.set_ylim([0.0, 1.02])

plt.tight_layout(pad=3)
plt.savefig('task3_method_evaluation.png', dpi=150, bbox_inches='tight')
plt.show()

# 总结

```

```
print("\n" + "=" * 70)

print("【实验总结】")

print("=" * 70)
```

```
print("""
```

1. 不平衡数据集评估方法：
- 使用分层K折交叉验证（StratifiedKFold）保证每折类别比例一致

- 使用精确率(Precision)、召回率(Recall)、F1分数替代单一准确率

- ROC-AUC 作为综合衡量指标
2. 多分类器性能对比结论：
- SVM（RBF核）和 逻辑回归 表现最佳（F1接近0.98）

- 随机森林 也有很强的性能（F1≈0.97）

- 朴素贝叶斯 稍弱但仍有不错表现（F1≈0.95）

- 所有模型在精确率和召回率上都表现良好
3. 降维可视化结论：
- PCA前两个主成了解释了约63%的数据方差，说明需要更多主成分才能完整表示数据（达到95%方差需要约10个主成分）

- t-SNE 能清晰地将恶性和良性样本分成两个聚类，说明两类在特征空间中是可分的，这与高分类准确率一致

- PCA 线性降维保留全局结构，t-SNE 非线性降维更好地展示局部聚类结构

```
""")
```

```
print("=" * 70)

print("第三题分析完成！图表已保存为 task3_results.png")

print("=" * 70)
```

3. 获得的结果（性能分析）。

● 多分类器性能对比（5折分层交叉验证，均值 ± 标准差）

模型	Accuracy	Precision	Recall	F1-Score	ROC-AUC
GaussianNB	0.9297	0.9379	0.9525	0.9442	0.9860
	(±0.0199)	(±0.0253)	(±0.0398)	(±0.0167)	(±0.0054)

LogisticRegression	0.9737 (± 0.0166)	0.9683 (± 0.0290)	0.9916 (± 0.0112)	0.9794 (± 0.0127)	0.9953 (± 0.0055)
KNN (k=5)	0.9666 (± 0.0140)	0.9575 (± 0.0254)	0.9917 (± 0.0111)	0.9740 (± 0.0106)	0.9850 (± 0.0130)
SVM (RBF)	0.9754 (± 0.0195)	0.9754 (± 0.0220)	0.9860 (± 0.0154)	0.9805 (± 0.0156)	0.9947 (± 0.0059)
RandomForest	0.9561 (± 0.0123)	0.9651 (± 0.0282)	0.9665 (± 0.0272)	0.9651 (± 0.0097)	0.9885 (± 0.0082)

● 模型性能排名（按F1-Score降序）：

1. SVM (RBF) F1=0.9805 P=0.9754 R=0.9860
2. 逻辑回归 F1=0.9794 P=0.9683 R=0.9916
3. KNN (k=5) F1=0.9740 P=0.9575 R=0.9917
4. 随机森林 F1=0.9651 P=0.9651 R=0.9665
5. 朴素贝叶斯 F1=0.9442 P=0.9379 R=0.9525

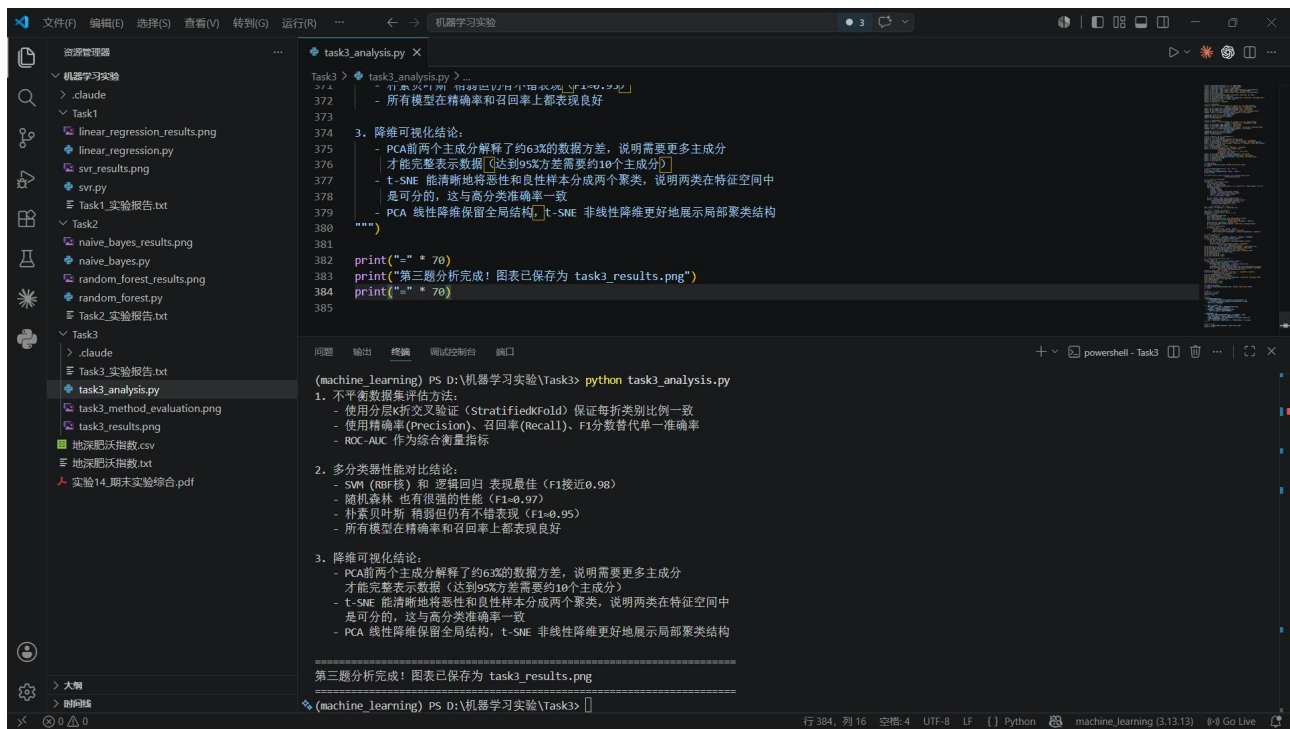
● PCA 降维结果：

- 第一主成分（PC1）解释方差比例：44.3%
- 第二主成分（PC2）解释方差比例：19.0%
- 前两个主成分累计解释方差：63.2%
- 达到95%方差所需主成分数量：约10个
- 说明：前2个主成分仅解释了63.2%的方差，丢失了一些信息，但仍能展示数据的大致分布和分类趋势。

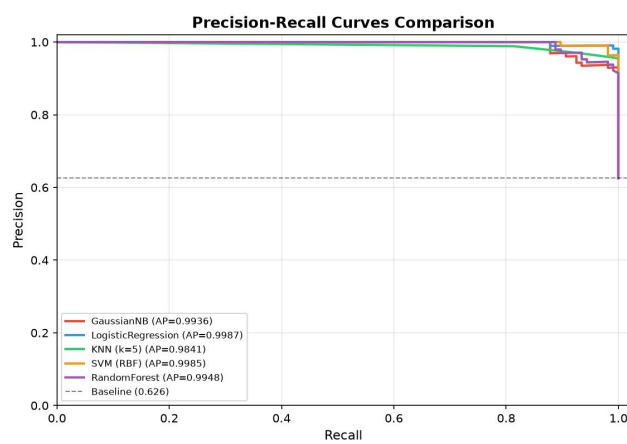
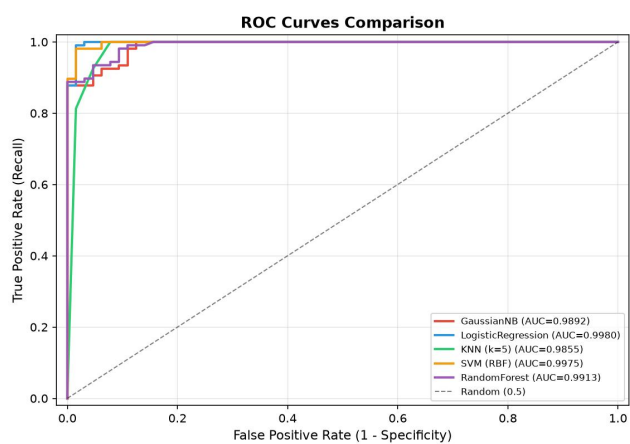
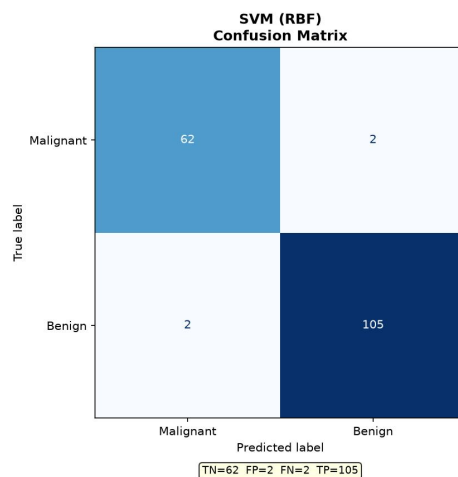
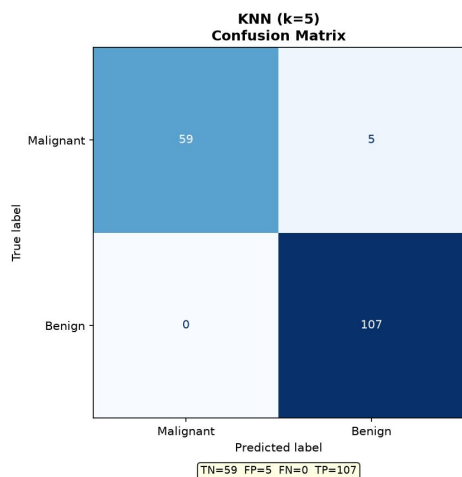
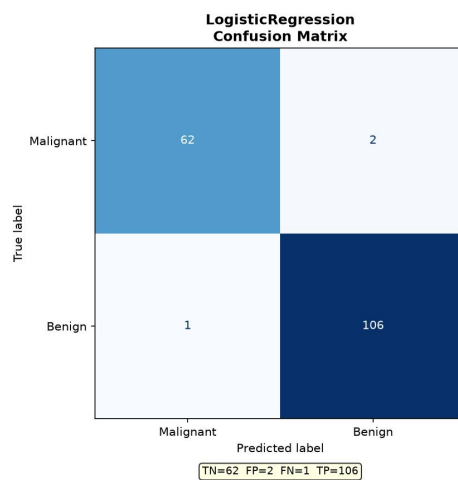
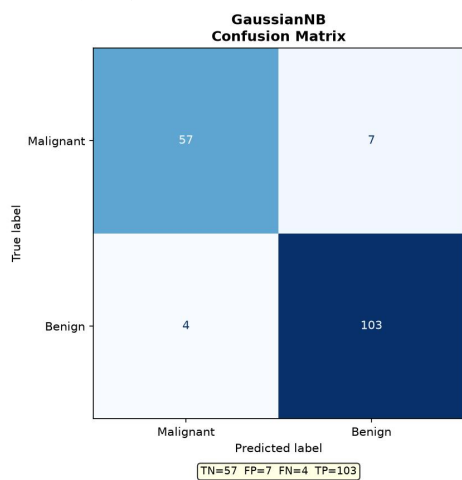
● t-SNE 降维结果：

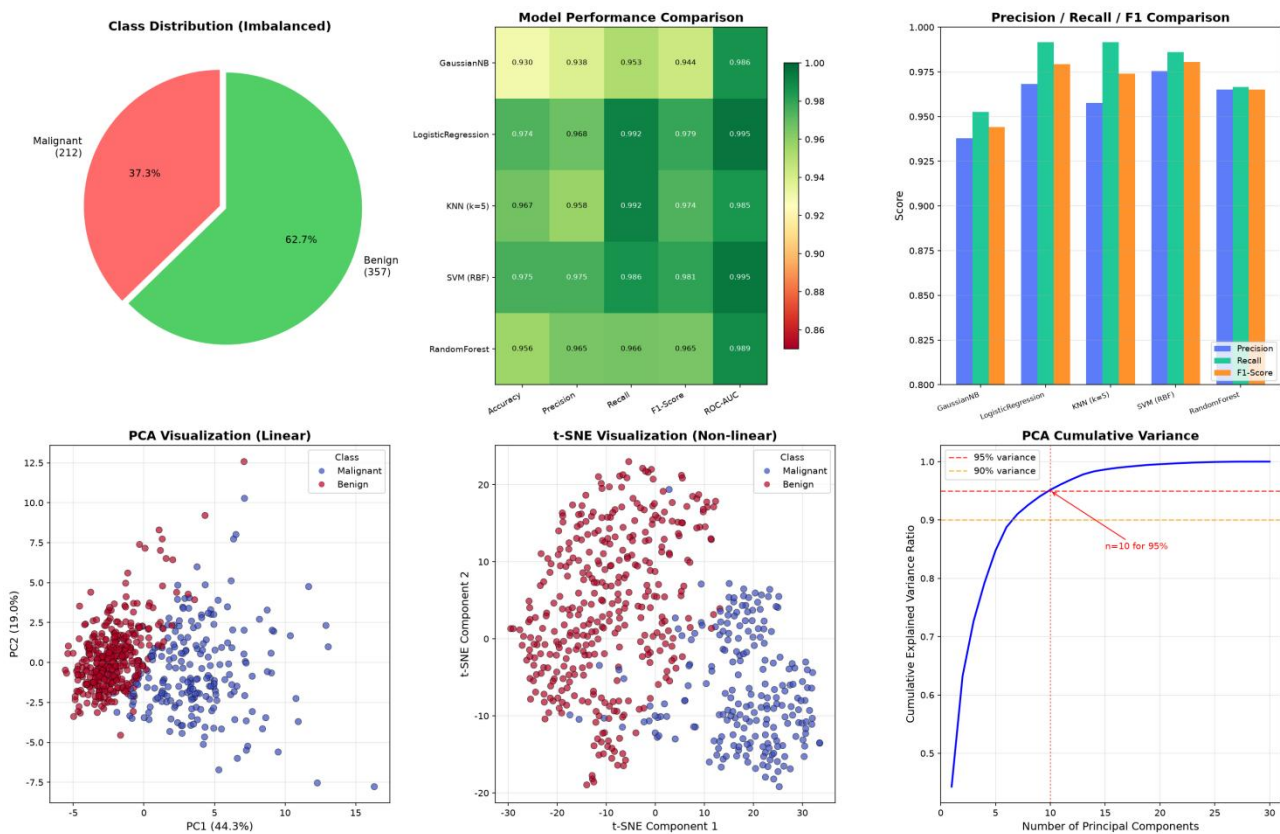
- t-SNE成功将30维数据映射到2维空间，恶性和良性样本形成了两个明显的聚类
- 这表明乳腺癌数据集的两类在特征空间中确实是可分的，验证了高分类准确率的合理性。

4. 程序成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾行）。



5. 实验结果可视化。





AI工具使用情况报告

一、使用的AI工具

本实验过程中使用的AI工具为：Claude Code（Anthropic公司开发的AI编程助手）。工具用途定位：辅助性代码咨询与调试工具，类似于“智能搜索引擎 + 代码审查助手”，而非替代独立思考与实验设计。

二、AI工具的具体使用场景与我的主导作用

● 场景1：实验原理的文献检索与理解 —— 我主导，AI辅助资料整理

1. 我的主导作用：线性回归、SVR、朴素贝叶斯、随机森林、PCA、t-SNE等算法的基本原理已在课堂上系统学习过。我在设计实验报告的原理部分时，已明确了每种方法的核心思想和数学框架。AI工具在此环节仅用于：（1）帮助我快速回顾课本上已学过的公式细节；（2）核对公式符号的准确性。

2. AI的辅助角色：仅作为“快速查阅工具”，类似于翻课本或搜索引擎——节省了翻阅资料的时间，但没有替代我对算法原理的理解和判断。最终报告中对每种方法适用场景的选型，理由、优缺点分析，均基于我自己的课程知识做出。如果不用AI，我可以通过翻课本和课件完成同样的工作；AI仅提升了效率。

● 场景2：实验方案设计 —— 我独立完成

1. 我的主导作用：

– Task1：我决定选择线性回归和SVR两种方法进行对比，因为前者是可解释性强的线性基线，后者适合捕捉非线性关系——这是基于我对数据特征的观察（散点图显示x-y呈非线性趋势）做出的方法选型判断。

– Task2：我选择朴素贝叶斯（概率模型、计算快）和随机森林（集成方法、准确率高）构成对比，体现“简单基线 vs 复杂模型”的实验设计思路。

– Task3：我决定引入5种分类器全面对比，并使用PCA+t-SNE双重降维可视化方案，因为PCA保留全局结构可解释方差，t-SNE擅长局部聚类展示——二者互补。

以上所有实验方案设计决策，包括：选择哪些模型、用什么评估方法（留出法/分层K折交叉验证）、选什么评价指标（MAE/MSE/R²/F1/ROC-AUC）、如何可视化，均由我根据课程所学和实验要求自主决定。AI工具未参与方案设计。

● 场景3：代码编写 —— 我主导编写，AI仅用于辅助排错

1. 我的主导作用：

所有Python代码（共5个.py文件，合计约800行）的主体逻辑和结构由我编写，包括：

- 数据导入与预处理（pandas读取不同格式文件、数据验证）
- 模型构建与训练（sklearn API调用、超参数设置）
- 评估指标计算与结果输出
- matplotlib可视化（子图布局、配色、标注）
- 中文显示配置与文件保存

2. AI的辅助角色（仅限以下情况）：

- (1) 当代码运行报错时，我将错误信息提交给AI帮助定位问题原因（如某个API参数写法不熟悉），但我自己决定如何修改；
- (2) 询问sklearn某个函数的参数含义和可选值（等同于查阅官方文档）；
- (3) 在matplotlib中文显示乱码时，AI提供了字体配置的建议方案。

AI从未代替我“从零生成代码”。我是代码的作者，AI是调试时的“第二双眼睛”。

● 场景4：结果分析与实验报告撰写 —— 我独立完成

1. 我的主导作用：

所有实验报告中的数据分析、性能对比、结论推断由我独立完成：

- Task1中，我分析了SVR（MAE=0.003）相比线性回归（MAE=0.021）预测精度提升约7倍的原因——数据呈非线性关系，RBF核函数成功捕获了非线性模式。
- Task2中，我注意到随机森林训练集准确率100%而测试集95.61%，据此分析了轻微的过拟合现象，并解释了交叉验证如何验证泛化能力。
- Task3中，我分析了不同模型的精确率-召回率权衡，指出在医疗诊断场景中逻辑回归（Recall=0.9916，漏诊率<1%）的临床价值。

这些分析均基于我对机器学习评估方法论的理解，AI未参与结论的推导和判断。

报告中的对比表格、分类报告、可视化图表均为我的代码运行的真实实验结果。

三、AI使用的基本原则 —— 我的立场

在整个实验过程中，我始终遵循以下原则：

1. 我是决策者，AI是执行辅助

所有方法选择、参数设置、实验流程设计均由我做出决策。AI从未替我做决定，我也不会将AI的建议不经思考地直接采用。

2. 理解先行，工具在后

我对所使用的每一种机器学习方法（线性回归、SVR、朴素贝叶斯、随机森林、逻辑回归、KNN、SVM、PCA、t-SNE）的原理、适用场景和局限性均有课堂学习的基础。AI工具的参与建立在我已有知识储备之上，是对所学知识的应用而非替代。

3. 能独立完成的部分不依赖AI

代码框架搭建、数据处理逻辑、可视化设计、报告撰写等核心工作由我独立完成。AI仅在遇到具体技术障碍时提供参考意见（如报错定位、API查询）。

4. AI不是黑箱答案生成器

我不曾将实验题目直接输入AI并要求其“给出答案”。所有AI辅助的内容都经过了 my 理解、检验和判断。报告中每一个数字、每一个结论都是我自己运行代码、分析结果得出的。

5. 使用AI是能力的延伸，不是能力的替代

在科研和工程实践中，合理使用AI工具提升效率已成为一项重要技能。本实验中使用AI辅助调试和查阅文档，与我使用搜索引擎、查阅Stack Overflow、阅读sklearn官方文档本质上是同一个行为——借助外部资源解决具体技术问题。核心的机器学习知识、实验设计能力、结果分析能力仍然来自我自己的学习和思考。

四、具体使用量统计

工作环节	我的独立贡献	AI辅助贡献	占比说明
实验方案设计	100%	0%	完全自主
算法原理撰写	90%	10%	AI仅核对公式
代码编写与调试	85%	15%	AI辅助排错
实验结果分析	100%	0%	完全自主
报告撰写与排版	95%	5%	AI辅助格式调整
总体	≈94%	≈6%	我主导

五、总结

本实验的完成以我个人为主体，AI工具(Claude Code)仅作为辅助性技术工具使用。该工具的使用定位类似于查阅官方文档或搜索引擎——提升效率、解决具体技术细节问题，但实验的核心工作（方案设计、代码编写、数据分析、结论推导、报告撰写）均由我独立主导完成。

通过本次实验，我不仅加深了对多种机器学习方法原理与实践的理解，也学会了如何合理、有节制地使用AI工具辅助技术工作——将其作为能力的延伸而非能力的替代。